

The
Pragmatic
Programmers

Haskell Brain Teasers

Exercise Your Mind



Rebecca Skinner

Series editor: *Miki Tebeka*

Development editor: *Michael Swaine*

Early Praise for Haskell Brain Teasers

Skinner's collection of fun and challenging brain teasers reminded me why I love learning languages in the first place. You'll see Haskell in a new light, so don't let your AI solve these for you!

► **Kevin Hoffman**

Author, Real-World Event Sourcing

Haskell Brain Teasers is a wonderful expose on some of the most fun and interesting parts of Haskell.

► **Matt Parsons**

,

If you've spent years writing Haskell and think you've internalized its quirks, these puzzles will prove you wrong in the best possible way. They sharpen intuition, reward careful reasoning, and remind you why the language remains endlessly fascinating.

► **Joel Stanley**

,

If you've learned the basics of Haskell, but then felt lost when looking at other's code, this book will give you the practical examples to understand and work with that code. You will get to see powerful patterns, thorough explanations, and alternative options. The book is a gold mine for the murky area of intermediate Haskell development.

► **Artem Chernyak**

Staff Software Engineer, Horizon Investments

I've read the C++ and C# Brain Teasers in this series, both excellent books, and I wanted to review Haskell Brain Teasers. I only know a little Haskell, so the puzzles were a great way to discover gaps in my knowledge. The book left me with lots to think about and many useful resources to learn more.

► **Frances Buontempo**

Independent Consultant, self



We've left this page blank to
make the page numbers the
same in the electronic and
paper books.

We tried just leaving it out,
but then people wrote us to
ask about the missing pages.

Anyway, Eddy the Gerbil
wanted to say "hello."

Haskell Brain Teasers

Exercise Your Mind

Rebecca Skinner

The Pragmatic Bookshelf

Dallas, Texas



See our complete catalog of hands-on, practical,
and Pragmatic content for software developers:

<https://pragprog.com>

Sales, volume licensing, and support:

support@pragprog.com

Derivative works, AI training and testing,
international translations, and other rights:

rights@pragprog.com

The team that produced this book includes:

Publisher: Dave Thomas

Executive Editor: Susannah Davidson

Development Editor: Michael Swaine

Copy Editor: Sean Tommasi

Copyright © 2026 The Pragmatic Programmers, LLC.

All rights reserved. No part of this publication may be reproduced by any means, nor may any derivative works be made from this publication, nor may this content be used to train or test an artificial intelligence system, without the prior consent of the publisher.

When we are aware that a term used in this book is claimed as a trademark, the designation is printed with an initial capital letter or in all capitals.

The Pragmatic Starter Kit, The Pragmatic Programmer, Pragmatic Programming, Pragmatic Bookshelf, PragProg, and the linking g device are trademarks of The Pragmatic Programmers, LLC.

Every precaution was taken in the preparation of this book. However, the publisher assumes no responsibility for errors or omissions or for damages that may result from the use of information (including program listings) contained herein.

ISBN-13: 979-8-88865-190-2

Encoded using recycled binary digits.

Book version: P1.1—April, 2026

Contents

Acknowledgments	vii
------------------------	-----

Introduction	ix
---------------------	----

Part I — Lazy Evaluation

Puzzle 1.	Looking for a Needle in a Stack of Infinities	3
Puzzle 2.	Haskell for Very Full Stack Developers	7
Puzzle 3.	Say Cheese	11
Puzzle 4.	Diving Head First into a Bottomless Pit	15
Puzzle 5.	The Least You Can Possibly Do	19
Puzzle 6.	Do You Even Lift?	23

Part II — Syntax and Language Extensions

Puzzle 7.	Oh Em Dash	29
Puzzle 8.	Playing with Matches	33
Puzzle 9.	Many Roads, One Destination	37
Puzzle 10.	A View of the Future	43
Puzzle 11.	FQDN - Fully Qualified do Notation	47

Part III — Type Classes

Puzzle 12.	Traversing Tuples	55
Puzzle 13.	A Slightly Off-White Hat	61
Puzzle 14.	If At First You Don't Succeed, Trie, Trie Again	67
Puzzle 15.	A Nice Recipe for Soup	71
Puzzle 16.	Field Theory	75

Part IV — The Type System

Puzzle 17.	Round-Trip Tickets	83
Puzzle 18.	A Grand Un-Unified Theory	89
Puzzle 19.	Non-Functional Dependencies	95
Puzzle 20.	Time Flies Like an Arrow	101

Acknowledgments

I would like to thank the many people whose contributions have helped make this book a reality. First and foremost I would like to thank my editor, Michael Swaine, copy editor Sean Tommasi, and Brain Teasers series editor Mikki Tebeka, along with Dave Thomas and everyone at The Pragmatic Programmers who worked with me to ship this book. I would also like to thank my technical reviewers: Matt Parsons, Frances Buontempo, Kevin Hoffman, and Jamis Buck. Their feedback helped me make this a book I'm truly proud of.

Haskell is a rich language and ecosystem that has benefited from the enormous talent and generosity of the contributors who make it what it is. To everyone who has contributed to the Haskell ecosystem, thank you for building a language I love to use, and for giving me something to write about.

Finally, I would like to thank my spouse, Ren Wilding, who has always believed in me, and my parrot, George, who has always believed in himself.

Introduction

Haskell's reputation is all about purity, safety, expressiveness, and power. It's a language admired by many but loved by few. For those of us who do love it, the joy comes from its flexibility and the ease of expressing our ideas without compromise.

Haskell's flexibility is one of its greatest strengths: A predictable and composable grammar gives developers the latitude to write code with nuance and style, and language extensions allow Haskell to continually raise the bar for what an industrial-strength language can do. At the same time, Haskell's flexibility can be an obstacle to fluency. When a language pragma changes the semantic meaning of your code or you first learn that “an expression” really means, yes, any expression, your reaction is more likely to be frustration than joy.

This book is here to help. As you work through short, focused puzzles, you'll gain a clear, intuitive understanding of Haskell and start to see it as a cohesive whole. And don't forget to stop and smell the rose trees.

Who This Book Is For

It's for you if you're a Haskell expert, are somewhat familiar with Haskell, or are just generally interested in functional programming.

How to Read This Book

Each puzzle begins with a small self-contained Haskell program intended to run with GHC 9.12 or later using the GHC2024 language version. All required imports and language extensions are included in the puzzles and examples.

When you read the puzzle, try to predict what will happen when you compile and run the program. For example, will it:

1. Run normally and print something to the screen?
2. Run slowly or use a lot of memory?
3. Fail to compile due to a syntax or type error?
4. Hang indefinitely in an infinite loop or deadlock?
5. Crash with a runtime exception?

Example Code and Tests

The sample code for this book includes standalone files for each puzzle, example code, unit and property tests, and a complete, reproducible Nix environment you can use to build and run the code. It's available on the Pragmatic Bookshelf website.¹ There, you can download the code, participate in discussions, and post issues if needed.

Puzzle Organization and Difficulty

You can work through the puzzles in any order. They're grouped thematically, based on which part of the language they focus on. For example, Part I focuses on challenges related to lazy evaluation, while puzzles in Part IV lean more into reasoning about types.

Puzzles in each part are organized roughly from easiest to hardest. Since difficulty varies depending on your personal experience and background, you might run across an easier puzzle that's challenging for you—or you might find a difficult puzzle quite easy. If you can't solve a puzzle, feel free to skip ahead and come back to it later.

Code Formatting

Although most of the code in this book aims to be readable idiomatic Haskell, some concessions have been made in the interest of length and formatting.

1. <http://pragprog.com/titles/haskellbt>

Part I

Lazy Evaluation

Laziness is great for programming languages, not so much for developers! In this first series of puzzles you'll need to exercise your mind to understand if and when lazy evaluation requires work. To solve these puzzles, you'll need to understand thunks, the relationship between pattern matching and evaluation, and how GHC implements common optimizations that rely on laziness. Try using `:print` in `ghci` as you work through these puzzles to see how expressions are evaluated interactively. If you find that a puzzle is too difficult, skip it and try again later or read the detailed explanation and try to come up with your own example that demonstrates the behavior.

Looking for a Needle in a Stack of Infinities

```
src/HaskellBrainTeasers/FirstFourDigitPrime.hs
module HaskellBrainTeasers.FirstFourDigitPrime where

primes :: [Int]
primes = sieve [2..]
  where
    sieve [] = []
    sieve (prime:candidates) =
      prime : sieve (filter divisible candidates)
      where
        divisible candidate = candidate `rem` prime /= 0

firstBigNumber :: [Int] -> Maybe Int
firstBigNumber =
  foldr findFirst Nothing
  where
    findFirst someNum isFound
      | length (show someNum) >= 4 = Just someNum
      | otherwise = isFound

main :: IO ()
main =
  case firstBigNumber primes of
    Nothing ->
      putStrLn "no big primes"
    Just aPrime ->
      putStrLn $ "the first big prime is " <> show aPrime
```

Guess the Output



Before moving on to the next page, try to guess the output. Do you think this program will complete at all? Will it complete quickly?

The program will output
the first big prime is 1009

Discussion

Thanks to lazy evaluation, this program not only runs—it finishes quickly.

Lazy evaluation is one of Haskell's most noteworthy features. Although it has a reputation for difficulty, you can see in this puzzle that lazy evaluation also lets us write code that efficiently deals with large and streaming datasets.

This puzzle uses lazy evaluation in two different ways: First, we lazily generate a list of all the prime numbers, calculating the next prime as needed. Second, we lazily search the list of primes, stopping as soon as we find an element that matches our criteria. Using laziness this way allows us to write straightforward code that still minimizes the amount of work we need to do.

Let's take a look at how laziness works in each of these cases.

Lazy List Generation

We start generating a list of primes by passing our sieve function an infinite list of all integers starting with two: `[2..]`. Haskell lists are ordinary linked lists that contain a reference to the value at the *head* of the list and a reference to the rest of the list, called the *tail*. Haskell's laziness lets us work with infinite lists since it allows us to use a reference to a *thunk* as a tail value. Thunks are placeholders for values we haven't computed. Instead of a value, a thunk holds references to the data and code we need to compute the value. If we do ask for the value of a thunk, we can compute it once and replace the thunk with the computed value.

We can see this working in practice in this modified version of our original sieve function. We start with the current prime number as the head of the list. The `tailThunk` will compute a new list that is itself composed of a new head value (the next prime number), a new tail thunk that will compute the next list, and so on.

```
src/HaskellBrainTeasers/FirstFourDigitPrime/Thunks.hs
```

```
sieve (prime:candidates) = prime : tailThunk
  where
    tailThunk = sieve (filter divisible candidates)
    divisible candidate = candidate `rem` prime /= 0
```

This works because each time we ask for the value of the tail of a list, we don't compute the *whole* tail but instead stop when we get to the next thunk. This lets us step one element at a time through the list.

Consuming Infinite Lists Lazily

Once we've created a list of primes, we need to search it. We can take advantage of lazy evaluation and the structure of lists to easily search even infinite lists using `foldr`. To understand how this works, let's look at an example of how we might write `foldr` ourselves:

`src/HaskellBrainTeasers/FirstFourDigitPrime/ExampleFoldr.hs`

```
Line 1 foldr :: (a -> b -> b) -> b -> [a] -> b
      2 foldr reduceWith accum input =
      3   case input of
      4     [] -> accum
      5     (a:as) ->
      6       reduceWith a $ foldr reduceWith accum as --
```

You can imagine `foldr` stepping through the list and, on line 6, calling `reduceWith` with the current element and the result of folding over the rest of the list. The subtle but important reason that this function allows us to work with infinite lists is that the recursive call to `foldr` is passed to `reduceWith` inside of a `thunk`.

Just as we could create an infinite list because the tail of the list was a `thunk` that would produce the next value on demand, we can search an infinite list because we consume the list one element at a time. We'll only recursively call `foldr` and start evaluating the next element of the list if the accumulator function (`reduceWith`) demands the accumulator value (`accum`).

In this puzzle, the accumulator function we're passing in is `findFirst`. This function first demands the value of the current list head (candidate) and checks to see if it's at least four digits. If it is, we return the value and never demand the accumulator (`isFound`), which means we never need to make another recursive call. If the current value is less than four digits, we return `isFound`, which is the recursive call that picks the next element from the list.

Using `foldr` this way can work with any infinite list and accumulator function so long as the infinite list only generates a finite number of values at a time and the accumulator function has at least one case where it doesn't demand that the accumulator be evaluated.

Further Reading

foldr in hackage

<https://hackage.haskell.org/package/base-4.21.0.0/docs/Prelude.html#v:foldr>

Sieve of Eratosthenes on Wikipedia

https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes

Puzzle 2

Haskell for Very Full Stack Developers

src/HaskellBrainTeasers/VeryFullStack.hs

```
module HaskellBrainTeasers.VeryFullStack where

data RoseTree a = RoseTree a [RoseTree a]

main :: IO ()
main =
  if "Maple" `elem` breadthFirst forest
  then putStrLn "Pancake time!"
  else putStrLn "No maple syrup!"
  where
    forest =
      RoseTree "Aspen"
        [ RoseTree "Baobab"
          [ RoseTree "Paw Paw" [], RoseTree "Basswood" []]
        , RoseTree "Cherry"
          [ RoseTree "Maple" [], error "boom!"]
        ]

breadthFirst :: RoseTree a -> [a]
breadthFirst (RoseTree a trees) = a : go trees
  where
    go [] = []
    go subtrees =
      let (vals, nextSubtrees) =
          foldr accumRoseTree ([], []) subtrees
      in vals <> go nextSubtrees
    accumRoseTree (RoseTree val subtrees) (vals, allSubtrees) =
      (val : vals, subtrees <> allSubtrees)
```

Guess the Output



Before moving on to the next page, try to guess the output. Do you think this program will complete at all? Will it complete quickly?

```
The program will fail with an error
boom!
HasCallStack backtrace:
  error, called at
src/HaskellBrainTeasers/VeryFullStack.hs:16:35 in
HaskellBrainTeasers.VeryFullStack
```

Discussion

In this puzzle, we run a breadth-first search over a `RoseTree`. A botanist may correctly point out that, technically speaking, a rose tree is not a type of tree but rather a woody shrub native to eastern North America. If you happen to encounter any roving botanists, you can safely assure them that our program is concerned with a data structure called a *rose tree*, where each node has a value and any number of children.

The rose tree in this puzzle contains the names of several botanist-approved trees. If we can find a maple tree, we'll be able to make pancake syrup. Without syrup, we'll be stuck with dry pancakes and nothing to distract the botanists. Unfortunately for our breakfast, we've put an error value in our forest. As we're trying to search through the tree, the lurking error value causes the program to throw an exception.

The behavior of this program can teach us something important about laziness in Haskell. Although the breadth-first search is lazy enough to work with an infinitely large tree, it won't work if any of the elements in the tree are malformed. This is shown in the figure at the top of the next page.

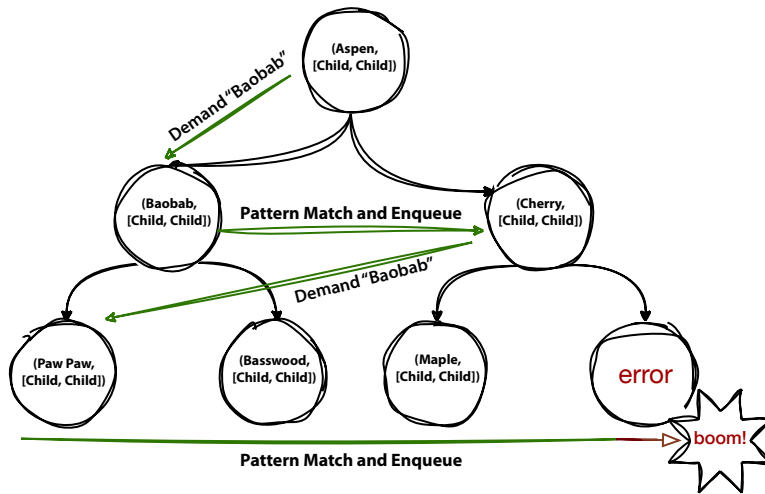
We can prove this to ourselves by trying to search an infinitely large tree:

```
src/HaskellBrainTeasers/VeryFullStack/InfiniteTree.hs
infiniteTree :: RoseTree Int
infiniteTree = go 0
  where go n = RoseTree n [go (n + 1)]

λ> 1009 `elem` breadthFirst infiniteTree
True
```

The fact that our code works for an infinitely large tree seems to imply that we can stop looking at values once we've found what we're looking for, but it turns out that the behavior is a bit more nuanced.

The breadth-first search takes a list of `RoseTrees` and returns the breadth-first traversal of those trees. It does this by walking through all of the trees at the current level and pattern matching on each `RoseTree` to pluck out the value of



the tree (a) and all of its subtrees (trees). When it's done, it adds all the tree values to the list and recursively moves down one level in the tree by combining all the subtrees at the current level.

This approach is lazy enough that we can support an infinitely deep tree, but it does require that we evaluate each `RoseTree` at the current level. We don't have to fully evaluate it. The tree's value and its list of subtrees can both be evaluated lazily, but pattern matching on `RoseTree` does require that we at least evaluate the value constructor. This is the root of the problem: Our raw error value is at the same level as the maple tree we're searching for, but it's not contained in a `RoseTree` constructor that we can safely pattern match against.

We can demonstrate this by replacing the raw error value with a `RoseTree` that contains error values instead:

```
src/HaskellBrainTeasers/VeryFullStack/AlternateError.hs
```

```
[ RoseTree "Maple" [], RoseTree (error "boom!") (error "another boom!") ]
```

This version of the program runs and works as expected:

The program will output
Pancake time!

This experiment is a good demonstration of the behavior of our program, but it's still not the most satisfying solution. No matter what we do, the program will crash if there's an error value and we're forced to look at it because we haven't yet found the value we're searching for. On the other hand, if we have found a value, it would be nice to avoid looking at any more rose trees. This would prevent us from failing in this example, but avoiding the extra work

would also mean a performance improvement in programs that take a lot of time or memory to calculate individual trees.

As it turns out, we can prevent unnecessary evaluation of trees and make the original program work correctly with a single-character change. In our breadth-first search, we just need to make a small change to the definition of `accumRoseTree`:

```
src/HaskellBrainTeasers/VeryFullStack/Fixed.hs
```

```
► accumRoseTree ~(RoseTree val subtrees) (vals, allSubtrees) =
    (val : vals, subtrees <> allSubtrees)
```

In this new version, we've turned the code that pattern matches on the `RoseTree` into an *irrefutable pattern* by adding a tilde (~) character. Unlike the ordinary pattern matches we were using before, *irrefutable patterns* are lazy and don't require that we evaluate the value we're matching against unless we demand the value that we're returning. In this case, we won't try to evaluate the error value unless we need to evaluate the value it should hold or we proceed to the next level in the tree and need to get its list of subtrees.

The trade-off is that an *irrefutable pattern match* is always considered successful, so it can't be used when we need to branch on whether a particular value matches the pattern. The following example uses an *irrefutable pattern* to match on just message:

```
src/HaskellBrainTeasers/VeryFullStack/BadPattern.hs
```

```
badPatternMatch :: Maybe String -> String
badPatternMatch ~(Just message) = message
badPatternMatch Nothing = "Nothing to see here"
```

Since this will always succeed and is considered an exhaustive match, we'll never fall through to the second case that handles a `Nothing` value. We could reorder the patterns and check for `Nothing` first, but doing so would demand the value and defeat the purpose of using an *irrefutable pattern*.

Irrefutable patterns fall into that particularly interesting subset of Haskell features that are both rarely used but indispensable when needed. They're often at their best when you want to implement lazy traversals of recursive and potentially infinite data structures.

Further Reading

Corecursion on Wikipedia

https://en.wikipedia.org/wiki/Corecursion#Tree_traversal

Case Expressions and Pattern Matching

<https://www.haskell.org/tutorial/patterns.html>

Puzzle 3

Say Cheese

src/HaskellBrainTeasers/SayCheese.hs

```
module HaskellBrainTeasers.SayCheese where
```

```
moons :: [(String, [String])]
```

```
moons =
```

```
  [ ("mercury",[])
    , ("venus",[])
    , ("earth",["Luna"])
    , ("mars",["Phobos", "Deimos"])
    , ("jupiter",["Io", "Europa", "Ganymede", "Callisto"])
    , ("saturn",["Mimas", "Enceladus", "Tethys", "Dione"])
    , ("uranus",["Oberon", "Titania", "Umbriel", "Ariel"])
    , ("neptune",["Proteus", "Larissa", "Galatea", "Despina"])
  ]
```

```
main :: IO ()
```

```
main = do
```

```
  let
```

```
    moonPrinter :: [(String, [String])] -> [(String, IO ())]
```

```
    moonPrinter [] = []
```

```
    moonPrinter ((planet, planetMoons):otherPlanets) =
```

```
      let
```

```
        message =
```

```
          if null planetMoons
```

```
          then putStrLn $ planet <> " has no moons"
```

```
          else putStrLn $ "The moons of "<>planet<>" are "<>show planetMoons
```

```
        in (planet, message) : moonPrinter otherPlanets
```

```
    pickJupiter message (planet, moonMessage)
```

```
      | planet == "jupiter" = moonMessage
```

```
      | otherwise = message
```

```
    foldl pickJupiter (putStrLn "starting search...") $ moonPrinter moons
```

Guess the Output



Before moving on to the next page, try to guess the output.

The program will output

The moons of jupiter are ["Io","Europa","Ganymede","Callisto"]

Discussion

At first glance, this program seems like it will walk through a list of planets and their most significant moons, printing each planet and its moons and stopping after printing out the moons of Jupiter. Surprisingly, though, when we run the program, we only get information about Jupiter and none of the planets before it. This is due to the common mistake of passing around `IO` actions as values without ever needing to evaluate them.

Haskell programs are typically run by evaluating `main`, which is an `IO` action with the type `IO ()`. In Haskell, we write pure functions that define and combine `IO` actions, which are in turn built by calling pure functions to create and combine smaller `IO` actions, and so on. We combine `IO` actions either directly by calling `(>>=)` or indirectly with `do` notation and functions that call `(>>=)` on our behalf. Remember that `do` notation is just syntactic sugar for `(>>=)`. We can always replace a `do` block with explicit calls to `(>>=)`. For example, `exampleDo` and `exampleBind` are equivalent here:

```
src/HaskellBrainTeasers/SayCheese/DoBindExample.hs
```

```
exampleDo = do
  cheese <- gouda
  brie cheese
  epoisses cheese

exampleBind =
  gouda
  >>= \cheese -> brie cheese
  >>= \_ -> epoisses cheese
```

When we're dealing with `IO` values, the `(>>=)` operator is *strict*: Whenever we combine two `IO` actions with `(>>=)`, either directly or in a `do` block, the first action always runs before the second. This is true even when we ignore the result. As an example, this program will print "This is Unnecessary!" before it prints "Hello, World."

```
src/HaskellBrainTeasers/SayCheese/Example1.hs
```

```
module HaskellBrainTeasers.SayCheese.Example1 where

unnecessary :: IO Int
unnecessary = do
  putStrLn "This is Unnecessary!"
  pure 5
```

```
main :: IO ()
main = do
  _unused <- unnecessary
  putStrLn "Hello, World"
```

Although the IO actions are combined strictly, IO actions themselves are ordinary values. We can bind them inside of let expressions, store them in data structures, pass them as arguments, and in general treat them like any other value. Having an IO action as a value doesn't run the action. If we change the example to use a let binding, it will no longer print *This is Unnecessary!* to the screen:

```
src/HaskellBrainTeasers/SayCheese/Example2.hs
{-# OPTIONS_GHC -Wno-unused-local-binds #-}
{-# OPTIONS_GHC -Wno-unused-matches #-}
module HaskellBrainTeasers.SayCheese.Example2 where

unnecessary :: IO Int
unnecessary = do
  putStrLn "This is Unnecessary!"
  pure 5

main :: IO ()
main = do
  let unused = unnecessary
  putStrLn "Hello, World"
```

When we evaluate IO actions, they always return a value. In some cases, as with `putStrLn`, the return value is the unit value and can easily be ignored. Other times, we may get useful values that we want to use in the larger overall IO action. An IO action may even return another IO action as a value. In such cases, returning an IO action does not bind it to the larger IO action. Unless done explicitly, the IO action we're returning will never be evaluated. Let's look at an example:

```
src/HaskellBrainTeasers/SayCheese/NestedIO.hs
nestedIO :: IO (IO String)
nestedIO = do
  let
    nested :: IO String
    nested = do
      putStrLn "I'm Nested!"
      pure "nested"
  putStrLn "Calculating nested IO..."
  pure nested

doesNotRunNestedIO :: IO ()
doesNotRunNestedIO = do
  nested <- nestedIO
  putStrLn "Done!"
```

Here, we create an IO action that gives us another IO action. In the case of `doesNotRunNestedIO`, we get the nested IO action as a value and do nothing with it, so it doesn't get evaluated and the function prints the following:

The program will output
Calculating nested IO...
Done!

`src/HaskellBrainTeasers/SayCheese/NestedIO.hs`

```
runsNestedIO :: IO ()
runsNestedIO = do
  nested <- nestedIO
  nested
  putStrLn "Done!"
```

In `runsNestedIO`, on the other hand, we use `nested` after binding it, so we run the nested IO action and see its output:

`src/HaskellBrainTeasers/SayCheese/NestedIO.hs`

```
runsNestedIO :: IO ()
runsNestedIO = do
  nested <- nestedIO
  nested
  putStrLn "Done!"
```

The program will output
Calculating nested IO...
I'm Nested!
Done!

The idea that we might have an IO action as a value that we never evaluate is the key to our puzzle. In our puzzle, `moonPrinter` returns a list of pairs, each consisting of the name of a planet and an IO action that prints out its moons. As we fold over the list, we disregard most of these IO actions. It's only when *jupiter* is found that we return an IO action that we actually evaluate—and so we only ever see the Jovian moons.

Further Reading

IO on Hackage

<https://hackage.haskell.org/package/base-4.21.0.0/docs/Prelude.html#t:IO>

Chapters 7 and 8 of Effective Haskell

<https://www.pragprog.com/titles/rshaskell/effective-haskell/>

Puzzle 4

Diving Head First into a Bottomless Pit

```
src/HaskellBrainTeasers/FriendOfAlice.hs
module HaskellBrainTeasers.FriendOfAlice where
import Data.List (intercalate)

data Person = Person { name :: String, friends :: [Person] }
    deriving (Show)

alice :: Person
alice = Person "Alice" [bob]

bob :: Person
bob = Person "Bob" [alice]

main :: IO ()
main = do
    let
        aliceFriends = name <$> friends alice
        bobFriends = name <$> friends bob
    putStrLn $ "Friends of Alice: " <> intercalate "," aliceFriends
    putStrLn $ "Friends of Bob: " <> intercalate "," bobFriends
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it finish running?

```
The program will output
Friends of Alice: Bob
Friends of Bob: Alice
```

Discussion

This puzzle demonstrates a technique called *knot tying*, where we take advantage of lazy evaluation to create cyclic data structures. Knot tying can seem confusing the first time you see it, but under the hood it works much like manually updating references in a strict mutable language.

Immutability and laziness are both important for knot tying, and it's important to develop an intuition for them if you want to tie the knot effectively. One of the most common obstacles to understanding laziness is the tendency to think about data structures strictly. You may have run into this problem while working on the puzzle—for example, when you read the definition of `alice`, you might have mentally filled in the record like this:

```
alice = Person "Alice" [Person "Bob" [Person "Alice" {...}]]
```

This doesn't get us far. Alice's only friend is Bob, whose only friend is Alice. They'll be best friends forever, and we'll never escape their friendship loop.

In languages with default mutability, we can get around this problem with references. We can use null values in place of circular references and update the references later to complete the knot:

```
js-examples/puzzle04.js
const alice = {name: "alice", friends: null}
const bob = {name: "bob", friends: [alice]}
alice.friends = [bob]
```

When you're accustomed to mutability, code like this feels natural because strictness and mutation go together like people with a circular friendship graph. Strict evaluation ensures that we can't create values such as `alice` or `bob` unless we can immediately provide all the fields we need. Mutability lets us start with friendless people and teach them to make friends later.

This wouldn't work well in Haskell. If we imagined a pure but strict version of Haskell, we'd find ourselves with a friendless version of `alice` who is admired by `bob`, but we couldn't update `alice` to reciprocate his unrequited friendship. We could create a second `alice`, but this doesn't quite work:

```
src/HaskellBrainTeasers/FriendOfAlice/WithUndefined.hs
```

```
alice = Person "Alice" []
bob = Person "Bob" [alice]
alice' = alice { friends = [bob] }
```

We end up with something that works with our puzzle, but it's not a truly cyclic data structure. Instead of `alice` and `bob` as mutual friends, we've introduced a doppelganger named `alice'` who is friends with `bob`, but `bob` is still friends with the original `alice`. We can see this in `ghci` by trying to print out a value for `bob`. Instead of looping forever, the program quickly completes by showing that `bob` is friends with a standoffish `alice`:

```
λ> bob
Person {name = "Bob", friends = [Person {name = "Alice", friends = []}]}
```

In the same way that strict evaluation required mutability so that we could update a value later with a reference we hadn't yet defined, we can see in this example that purity is pushing us toward laziness. If we can't update `alice` later to add a fully constructed value of `bob` to her list of friends, the alternative is to use a lazily computed placeholder for `bob` that isn't available now but will be when we need him.

Laziness isn't strictly required (pun intended). There are strict pure languages that let us represent cyclic data, but they require that we either use a localized form of mutability (such as `IO`) or encode the cycles indirectly. Lazy impure languages are a different story. Since mutation or side effects would only happen on demand, we'd end up with a language popular among the dreaming gods in R'lyeh—but too hard to reason about for most practical uses.

These examples speak to an important property of programming languages: Strictness and mutability work well together and laziness and purity work well together, but pure strict languages can't easily represent things like cyclic data structures, and lazy impure languages would make it impossible to reason about when things would change.

On the surface, it might seem like the pure lazy approach would be inefficient, but it turns out that the runtime behavior is remarkably similar to the strict mutable code we wrote by hand. In the strict version, `alice` contains a reference that starts out `null`. We run some code to create `bob` and then update the reference in `alice` to point to that new value. In our Haskell program, we don't have to manage references directly. Instead, `alice` and `bob` are both stored as *thunks*. A thunk contains a reference to a result that is initially `null`, a reference to some code that we can run to compute a result, and references to any data that was in scope when the thunk was created. Since `alice` and `bob` are both

static top-level thunks not created dynamically, they're said to be *constant applicative forms*, which is almost always abbreviated *CAFs*.

The first time we need to get a value from a thunk, the runtime will run the code and then update the result so that it contains a reference to the computed value. Afterwards, whenever we need the value from the thunk, we can use the already computed result. The two key differences between thunks and functions are that thunks never have arguments and they can be updated. Functions always have at least one argument and have to be computed every time.

At runtime, we start by printing out the names of Alice's friends, then Bob's friends. We evaluate the *alice* thunk, which gives us a *Person* constructor with thunks for *name* and *friends*. We then update the result reference in the *alice* thunk to this new *Person* value so that we don't need to recompute it. Next, we need to evaluate the *friends* field, which is a list that contains a thunk for *bob*. We evaluate the *bob* thunk, getting another *Person* constructor with its own *name* and *friends* thunks, and we update the result field of *bob* to hold that new value. That's enough information to print the names of Alice's friends and stop evaluating thunks. Next, we need to print a list of Bob's friends. We've already computed the *bob* thunk and have a *Person* record, but we haven't looked at the *friends* field. It contains a list with an *alice* thunk. When we look at that thunk, we see that it's already been updated with a result pointing to Alice's *Person* record, so we can grab the *name* field from that record and stop evaluating thunks.

The result of this process is that the runtime updates to the result field in each thunk end up doing the same thing that manually updating the *friends* field reference did in our JavaScript example. But in Haskell, the process is handled automatically. We still end up with a compact pair of values that each contain a reference to the other, and we still deferred filling in references, but laziness gave us a simpler conceptual model to use. Even with more advanced knot tying examples, the underlying mechanism for knot tying still relies on thunks, and the thunk update process creates sensible data structures in memory while saving us the risk of explicitly dealing with pointers and null values.

Further Reading

Tying the knot on the Haskell Wiki

https://wiki.haskell.org/Tying_the_Knot

Mutual Recursion

https://en.wikipedia.org/wiki/Mutual_recursion

Puzzle 5

The Least You Can Possibly Do

`src/HaskellBrainTeasers/PointlesslyFixed.hs`

```
module HaskellBrainTeasers.PointlesslyFixed where
import Data.Function
main :: IO ()
main = print (go 12)
  where go = take<*>fix(((*)<*>).(.succ))
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it finish running?

The program will output
[12,13,14,15,16,17,18,19,20,21,22,23]

Discussion

This puzzle defines an absurdly pointfree function that returns n numbers, starting at n . We could have written this more clearly as follows:

```
src/HaskellBrainTeasers/PointlesslyFixed/PointfulRefactor.hs
refactored count = take count (startingAt count)
  where startingAt n = n : startingAt (n + 1)
```

Although this second version of the code is undoubtedly easier to read in this example, with a bit of practice you can learn to convert between pointfree and pointful representations of code. You're probably already familiar with function application and partial application. The code in this puzzle uses both, but it also takes advantage of two less common and more difficult to understand tools. It uses the Applicative operator to combine functions and the `fix` function to create anonymous recursive functions.

In the rest of this discussion, we'll look at each of these in more detail to help you understand how they work and are used in pointfree code.

The Applicative Instance for Functions

The Applicative type class is widely used in Haskell, and you're probably already familiar with the `(<*>)` operator in contexts other than directly combining functions. In spite of any familiarity in other circumstances, people often find the use of `(<*>)` with functions to be particularly challenging. Thankfully, there's an easy way to translate between `(<*>)` and more familiar syntax:

```
src/HaskellBrainTeasers/PointlesslyFixed/PointfulRefactor.hs
withApplicative = f <*> g
withoutApplicative a = f a (g a)
```

Although the translation is easy to memorize, it might sometimes feel unnatural to use `(<*>)` with functions. After all, it's much more common to use the Applicative instance for things such as `State` and parsers. It's helpful to remember that `State` and state-like parsers are typically implemented as functions. Their Applicative instance often looks something like this:

```
src/HaskellBrainTeasers/PointlesslyFixed/StateDemo.hs
State f <*> State g = State $ \s ->
  let (fa, s') = f s; (a, s'') = g s' in (fa a, s'')
```

If we imagine an ordinary function as a sort of trivial stateful function that doesn't actually have any state, we might write an instance for it like this:

```
src/HaskellBrainTeasers/PointlesslyFixed/FunctionDemo.hs
newtype Function a b = Function (a -> (b, ()))
Function f <*> Function g = Function $ \s ->
  let (fa, _) = f s; (a, _) = g s in (fa a, ())
```

Comparing this to an Applicative instance for State, we can see how it matches up with `f a (g a)` as the definition of `(<*>)` for ordinary functions.

The Least Fixed Point

The documentation for `fix` says that it finds the “least fixed point” of a function, which is a value where a function's input is equal to its output. But we're not restricted to using `fix` with functions that have a well-defined fixed point value. We can use `fix` to write any function that we could write with direct recursion. The key difference is that `fix` passes the recursive call to our function as an argument, and with direct recursion we make the recursive call by having a function call itself by name.

Like any other recursive code, `fix` works as long as the recursive call is lazy. In our puzzle, for example, the recursive call is made in the tail of a list. Our call to `take` stops demanding values once it gets the required number of elements from the list, allowing the recursion to terminate.

```
src/HaskellBrainTeasers/PointlesslyFixed/PointfulRefactor.hs
findFixedEven :: (Int -> Int) -> Int -> Int
findFixedEven recursiveCall n =
  if even n then n else recursiveCall (n + 1)
```

This example makes it easy to see how `fix` turns recursion into an argument we can call. The `recursiveCall` argument is the recursion that `fix` gives us, and `n` is the value at the current point. Thanks to laziness, if we never call `findFixed`, then `fix` will terminate and give us back a concrete value.

Unfortunately, even when we see how `fix` is implemented, the type can still throw us for a loop. We've just seen how `fix` calls our function with a function we can use to make a recursive call and a value. That makes sense right up until we look at how `fix` is defined:

```
fix :: (a -> a) -> a
fix f = let x = f x in x
```

At first glance, the definition of `fix` can be a bit intimidating. You can see from the `let` bindings where the recursion comes from, but it's hard to reconcile the type and definition of `fix` with the fact that we're passing it something that

takes two arguments. Thankfully, it makes a lot more sense if we actually substitute the type of `a` with a concrete example from `findFixedEven`:

```
findFixedEven :: (Int -> Int) -> (Int -> Int)
fix findFixedEven :: Int -> Int
```

Looking at the code this way, it's easier to see that `fix findFixedEven` returns a function instead of a fixed point value, as you might initially expect. The function it returns will give us the least fixed point value starting at its argument.

Let's put this all together. We're passing `findFixedEven` into `fix`, so we could write its definition like this:

```
fix findFixedEven =
  let recursiveCall = findFixedEven recursiveCall
  in recursiveCall
```

Substituting the values makes it easier to see how we're getting a recursive function, but it still may not be immediately obvious how we're passing the value through all these recursive calls. Remember that `recursiveCall` is a function from `Int` to `Int`.

The very first call we make to `recursiveCall` will be made with our initial number. That will get bound to `n` in the body of `findFixedEven`, where we return `n` directly if it's even or call `recursiveCall` and pass it the next value, `n+1`. In practice, this is the same as passing a new value in a directly recursive style. The only difference is that, instead of calling `findFixedEven` directly, we recurse indirectly through `recursiveCall`.

This indirect recursion is how `fix` allows us to write anonymous recursive functions. Since we're not directly referencing the function we want to call recursively, we don't have to name it—although, of course, we can if we want to. Once you understand how `fix` works, you can translate back and forth between named and anonymous recursive functions:

```
anonymous = fix something
named n = something named n
```

Further Reading

An extended tutorial on `fix`

<https://rebeccaskinner.net/posts/2021-06-09-getting-to-the-fixed-point.html>

Haddock documentation on `fix`

<https://hackage.haskell.org/package/base-4.21.0.0/docs/Data-Function.html#v:fix>

Do You Even Lift?

```
src/HaskellBrainTeasers/UnsafeToTheMax.hs
module HaskellBrainTeasers.UnsafeToTheMax where
import System.IO.Unsafe
import Data.IORef
import Data.Foldable
import Control.Monad

unsafeMax :: [Int] -> IO Int
unsafeMax vals = do
  for_ vals $ \val -> do
    currentMax <- readIORef maxRef
    when (val > currentMax) $
      writeIORef maxRef val
    readIORef maxRef
  where
    maxRef = unsafePerformIO $ newIORef 0

main :: IO ()
main = do
  four <- unsafeMax [4,3,2]
  three <- unsafeMax [3,2,1]
  two <- unsafeMax [2,1,0]
  print [four, three, two]
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it finish running?

When built with optimizations, the program outputs
[4,4,4]

When built without optimizations, the program outputs
[4,3,2]

Discussion

This puzzle illustrates the dangers of `unsafePerformIO`. Although the program type checks, its runtime behavior depends on how it's built. To fix the puzzle, we'll need to understand how `unsafePerformIO` works, when it doesn't, and how to guard against things that might cause our program to behave unpredictably.

Impure Haskell

Although Haskell has a reputation for strictly enforcing pure functional programming, there are escape hatches that allow you to evaluate IO actions in pure code. These functions aren't just a novelty: `unsafePerformIO` and its cousins `unsafeDupablePerformIO` and `unsafeInterleaveIO` solve real problems and can be used to write safe and easy-to-use code. Even the amusingly named `accursedUnutterablePerformIO` is more than just a trap for the incautious developer. Still, useful or not, this puzzle demonstrates that these functions have rightfully earned the *unsafe* part of their names.

To use `unsafePerformIO` safely you need to ensure the following:

- The IO action doesn't interact with the external environment
- The call can't be moved elsewhere by the compiler
- You don't use `unsafePerformIO` to get a polymorphic value

Avoiding the External Environment

`unsafePerformIO` is best used for code that's pure from the perspective of the consumer. If you're careful, you can use `unsafePerformIO` to safely acquire and initialize resources from the system—for example, by creating an `IORef` or `MVar`. Since these resources are created on demand and not shared, creating a new one isn't a visible side effect anywhere else in your code.

The difference between what's internal to your program and what's part of the external environment often comes down to personal judgment and risk tolerance rather than clear objective guidelines. Reading or writing to ordinary files is risky because anything else inside or outside your program could change or delete a file at any time, and the exact time that you'll read the

contents of the file are nondeterministic. Initializing your program with the contents of a read-only file is probably safer but still not guaranteed to work in the presence of a user with `sudo` access and a grudge. Opening a new log file at program startup is safer still, unless the file system has disappeared or our sudoer got `chmod`-happy. `unsafePerformIO` is, in the end, not safe, and it's up to you as a developer to decide when to use it.

Avoiding Relocation

One of the biggest risks when using `unsafePerformIO` is that our program might behave unexpectedly because of optimizations GHC makes that are safe for pure functions but break down when we introduce side effects. This usually happens because GHC has changed when or how often side effects get evaluated. There are three causes for this: inlining, let floating, and common subexpression elimination (CSE).

When GHC inlines code, you'll run side effects more often than anticipated. Instead of running an IO action once to allocate and initialize a resource, your program will create a new resource each time the value is referenced. In common cases, such as using `unsafePerformIO` to create an `IORef` or an `MVar`, it can look as if values aren't being set when they should be. What's actually happening is that each time you reference the `IORef` or `MVar`, you're getting a newly initialized value instead of a shared value that you can write to and read from the way you expect.

You can disable inlining with the `NOINLINE` pragma. This tells GHC that it shouldn't inline an expression even when its heuristics tell it that it should. You should almost always use `NOINLINE` whenever you use `unsafePerformIO` with a top-level identifier.

Whereas inlining can cause your side effects to be run more often than intended, let floating can cause them to be run less often than intended. GHC uses let floating to move expressions up to a higher scope when it notices that they could be efficiently reused. If an `MVar` or `IORef` is relocated because of let floating, it may look like they're being initialized with an incorrect value. This happens because you're getting a reference shared with whatever value happened to be last written instead of a newly initialized reference. let floating can be hard to predict, and you can't disable it on a per-function basis, but it's good practice to disable it in any module that uses `unsafePerformIO` by adding `{-# OPTIONS_GHC -fno-full-laziness #-}` to the top of your module declaration.

The last optimization that could break `unsafePerformIO` is called *common subexpression elimination*, or CSE. This happens when the compiler recognizes that

the same expression is used multiple times and combines them. The practical result is that you run your side effects less often than anticipated, similar to let floating. GHC only does CSE in a few specific circumstances, but it can be hard to predict. To avoid potential errors, you should add `-fno-cse` to the `OPTIONS_GHC` for your module, alongside `-fno-full-laziness`.

Avoiding Polymorphic Values

One final problem we need to be aware of when using `unsafePerformIO` is that it can let us write type-unsafe code. Without `unsafePerformIO`, for example, we can never get a hold of an `IORef` value where the type of value referenced can't be inferred by the compiler. Using `unsafePerformIO`, we can get hold of an actual polymorphic `IORef` value that could hold any type. This allows us to write an unsafe coercion function that we can use to turn a value of any type into a value of any other type, potentially resulting in a runtime crash:

```
src/HaskellBrainTeasers/UnsafeToTheMax/UnsafeCoerce.hs
```

```
{-# NOINLINE ref #-}
ref :: IORef a
ref = unsafePerformIO $ newIORef undefined

unsafeCoerce :: a -> b
unsafeCoerce from = unsafePerformIO $
  writeIORef ref from >> readIORef ref
```

Further Reading

INLINE pragma in the GHC User's Guide

https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/pragmas.html#inline-pragma

unsafePerformIO on Hackage

<https://hackage.haskell.org/package/base-4.21.0.0/docs/System-IO-Unsafe.html#v:unsafePerformIO>

accursedUnutterablePerformIO documentation

<https://hackage.haskell.org/package/bytestring-0.12.2.0/docs/Data-ByteString-Internal.html#v:accursedUnutterablePerformIO>

Referential Transparency on the Haskell wiki

https://wiki.haskell.org/index.php?title=Referential_transparency

Part II

Syntax and Language Extensions

Syntax and Semantics and Grammar, Oh My! This series of puzzles will challenge you to understand the ins and outs of Haskell's syntax, from Haskell98 oddities to modern GHC features such as or-patterns and qualified do notation. Throughout these puzzles, you'll make heavy use of language extensions that change how Haskell behaves, so be sure to pay attention to which extensions are enabled, both in your source files and in ghci. If you find that a puzzle is too difficult, skip it and try again later or read the detailed explanation and try to come up with your own example that demonstrates the behavior.

Oh Em Dash

```
src/HaskellBrainTeasers/OhEmDash.hs
```

```
module HaskellBrainTeasers.OhEmDash where
```

```
isIdentity :: (Int -> Int) -> String
```

```
isIdentity f = show $ or [f n == n | n <- [0..10]]
```

```
main :: IO ()
```

```
main = do
```

```
    putStrLn $ "Are These Identities?"
```

```
    putStrLn $ "*1 " <> isIdentity (* 1)
```

```
    putStrLn $ "+0 " <> isIdentity (+ 0)
```

```
    putStrLn $ "-0 " <> isIdentity (- 0)
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

```

The program will fail to compile with an error:
/tmp/OhEmDashXXX122964-0.hs:11:35: error: [GHC-39999]
    " No instance for 'Num (Int -> Int)'
        arising from a use of syntactic negation
        (maybe you haven't applied a function to enough arguments?)
    " In the first argument of 'isIdentity', namely '(- 0)'
        In the second argument of '(<>)', namely 'isIdentity (- 0)'
        In the second argument of '($)', namely '"-0 " <> isIdentity (- 0)'
11 |   putStrLn $ "-0 " <> isIdentity (- 0)
    |                                     ^^^

```

Discussion

This puzzle demonstrates an inconsistency in Haskell’s syntax for partially applying operators.

Haskell gives us two notations for calling functions: prefix and infix.

Prefix notation is what you might think of as the “ordinary” way of calling functions in Haskell. Using prefix notation, we start with the name of the function and then apply arguments. Since Haskell functions are curried by default, we can do partial application by leaving out some arguments:

```

src/HaskellBrainTeasers/OhEmDash.hs
appendThree a b c = a <> b <> c

fullyApplied = appendThree "1" "2" "3"
applyTwo = appendThree "1" "2"
applyOne = appendThree "1"

```

You can only partially apply prefix function arguments in the order they’re defined. If we want to apply only the second or third argument, we need to introduce variables to fill in the arguments we don’t want to apply:

```

prependToThree a b = appendThree a b "3"

```

Unlike named functions, operators are infix by default. Infix functions take two arguments. The first is the left-hand operand and the second is the right-hand operand.

Although named functions use prefix notation by default and operators use infix notation by default, we’re free to use named functions as infix functions and operators as prefix functions. Surrounding an operator in parentheses allows you to use it as a prefix function, and surrounding a named function with backticks lets you use it as an infix function. Let’s look at an example:

```
src/HaskellBrainTeasers/OhEmDash.hs
```

```
additionInfix = 1 + 2 + 3
additionPrefix = (+) ((+) 1 2) 3

maxInfix = 1 `max` 2
maxPrefix = max 1 2
```

In most cases, prefix and infix notation work as expected, and the code we end up with is intuitive to read and write. One edge case is partial application. Syntactically, operators present a bit of a challenge for partial application because, unlike ordinary curried functions that always take one argument, operators are binary functions that take two arguments. Practically speaking, Haskell functions are often designed with partial application in mind and conveniently put the arguments you're most likely to want to partially apply first in the argument list. But for operators, it's rarely clear which operand someone is most likely to partially apply.

Haskell's solution to this is *operator sections*. Operator sections are a special syntax that allow you to partially apply the left or right operand of any infix operator. We used operator sections in our puzzle, but we can use them with any operator—for example, `(/2)` or `("hello" <>)`.

Operator sections are typically convenient and easy to use, but they leave us with a grammatical ambiguity: Since the `(-)` operator is used for both subtraction and writing negative numbers, the compiler can't differentiate between negation and subtraction when it sees an expression such as `(-2)`.

Haskell's grammar directly addresses this: A minus sign with an expression to the left is always treated as subtraction; otherwise, it's treated as negation. This removes ambiguity, but it leads to a couple of quirks. First, we can't create an operator section to partially apply the second argument to subtraction. Instead, expressions such as `(-0)` will always be treated as negative literals. Second, whenever we pass a negative value into a function, we have to put parentheses around it to avoid the minus operator being treated as subtraction. The table at the top of the next page shows how the presence or absence of a left-hand argument affects how the code is parsed. You can also see how desugaring `(-)` is done before type checking.

In the first part of our example, we get a fairly verbose compilation error. The crux of the problem is that the compiler is treating the expression `max -5 0` as though we'd written `(max - 5) 0` and then reasonably tells us that trying to subtract five from a function doesn't make sense. Our second example works because, with the added parentheses, `(-5)` is treated as an ordinary negative number.

Oh Em Dash					
Expression	Haskell 2010	LexicalNegation	Expression Left of (-)?	Space Before (-)?	Space After (-)?
(-0)	negate 0	fromIntegral (-0)	 No	 No	 No
(- 0)	negate 0	\x -> subtract x 0	 No	 No	 Yes
(-0)	negate 0	fromIntegral (-0)	 No	 Yes	 No
(- 0)	negate 0	\x -> subtract x 0	 No	 Yes	 Yes
(x - 0)	subtract x 0	subtract x 0	 Yes	 Yes	 Yes
(x-0)	subtract x 0	subtract x 0	 Yes	 No	 No
(x- 0)	subtract x 0	subtract x 0	 Yes	 No	 Yes
(x -0)	subtract x 0	x \$ fromIntegral (-0)	 Yes	 Yes	 No

The Haskell language standard recognized the limitations caused by this grammar rule. Adding parentheses addresses the problem of passing negative numbers into a function. For partial application, Prelude provides the `subtract` function that you can use in lieu of an operator section. We can use it to fix the code in our puzzle:

```
putStrLn $ "-0 " <> isIdentity (subtract 0)
```

As of GHC 9, we have a second option: `LexicalNegation`. This language extension uses whitespace to differentiate between subtraction and negation. With this extension enabled, our puzzle will behave as expected because the space in `(- 0)` ensures that `(-)` is treated as subtraction.

Further Reading

Operator sections in the Haskell Report

<https://www.haskell.org/onlinereport/exps.html#sections>

Unary operator on the Haskell wiki

https://wiki.haskell.org/Unary_operator

Lexical negation in the GHC User's Guide

https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/lexical_negation.html

Playing with Matches

```
src/HaskellBrainTeasers/PlayingWithMatches.hs
module HaskellBrainTeasers.PlayingWithMatches where
import Data.List (find)

-- | A Pythagorean Triple
type Triple = (Int, Int, Int)

findTriples :: [Int] -> [Int] -> Maybe Triple -> Maybe Triple
findTriples sides hypotenuses fallback
  | (a:b:_) <- sides
  , let matches c = a^2 + b^2 == c^2
  , Just c <- find matches hypotenuses = Just (a,b,c)
  | (_:rest) <- sides = findTriples rest hypotenuses fallback
  | Just (a,b,c) <- fallback
  , a^2 + b^2 == c^2 = fallback
  | otherwise = Nothing

main :: IO ()
main = print $ findTriples [1..10] [1..10] (error "boom")
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

The program will output
Just (3,4,5)

Discussion

Pythagoras was a well-known cult leader and an avowed enemy of all things bean related. He also did some mathematics, including work on Pythagorean triples, which are sets of three whole numbers that form the sides of a right triangle.

In our puzzle, we're playing with pattern matches while searching through a list of triangle sides and hypotenuses to see if we can construct a valid triple. If not, we'll look to our fallback value for an answer.

Since our puzzle is pattern matching on fallback, you might expect it to go boom when we pass in an error value. Luckily for Pythagoras, in this puzzle we're using *pattern guards* to search for a triple, allowing us to safely bypass the fallback and avoid an explosion as long as we find a match.

Although pattern guards have been around for a while, they can still cause confusion. Basic pattern guards are common enough in everyday Haskell codebases, but some of their more advanced uses may surprise even seasoned Haskellers.

Standard Guards

Standard guard clauses in Haskell let us select a definition for our function based on some criteria. Each guard associates some boolean expression with a function body, and the guards are evaluated top to bottom until an expression returns `True`. For example, `relationship` will return a relationship between three numbers: first by checking to see if the first two arguments are equal, then if the first three arguments appear in increasing order, and finally if they're a Pythagorean triple. If none of those are true, it returns "unrelated":

```
src/HaskellBrainTeasers/PlayingWithMatches/Examples.hs
relationship :: Int -> Int -> Int -> String
relationship a b c
  | a == b = "a and b match"
  | (c == b + 1) && (b == a + 1) = "increasing"
  | a^2 + b^2 == c^2 = "pythagorean triple"
  | otherwise = "unrelated"
```

Guards are evaluated from top to bottom lazily. We can see this for ourselves in `ghci`:

```
λ (relationship 1 1 (error "boom"), relationship 3 4 5)
("a and b match", "increasing")
```

The first call to `relationship` returns “a and b match” rather than failing, because we satisfy the first guard and never need to evaluate any subsequent guards that would force the error. This is the same reason our puzzle was able to return something useful even though our default value was an error.

Ordinary guard clauses contain boolean expressions, meaning any expression of the type `Bool`. Since any boolean expression is allowed, you can write relatively complex logic, including `let` expressions, `if` then `else`, and even case expressions, directly in a guard. Here’s an example:

```
src/HaskellBrainTeasers/PlayingWithMatches/NoPattern.hs
relationship :: Int -> Int -> Int -> String
relationship a b c
  | case a - b of
    0 -> True
    _otherwise -> False
  = "a and b match"
  | if b == a + 1
    then c == b + 1
    else False = "increasing"
  | let c' = a^2 + b^2 in c' == c = "pythagorean triple"
  | otherwise = "something else"
```

Pattern Guards

Pattern guards give us a more grounded way to achieve this same expressive power without sacrificing readability. Instead of associating each guard with a single binary expression, pattern guards allow us to provide a comma-separated list of *qualifiers*. Each qualifier can be one of the following:

- A boolean expression that matches if it returns `True`
- A pattern that matches if the pattern matches
- A `let` expression, which always matches

A guard matches if all its qualifiers match. Guards are still evaluated lazily from top to bottom. When a guard is evaluated, each qualifier is checked in order until one fails or they’ve all matched. Qualifiers are evaluated like any other expression and can still contain laziness:

```
src/HaskellBrainTeasers/PlayingWithMatches/Examples.hs
lazyGuard :: Maybe Int -> Maybe Int -> String
lazyGuard maybeA maybeB
  | Just a <- maybeA, Just b <- maybeB
  , (a > 10) || (b > 10) = "Greater than 10"
  | otherwise = "not that great"
```

In this example, we're pattern matching on both of the `Maybe` values we pass in. If we have values for both, then we check to see if at least one of them is larger than ten. If we pass in `Just (error "boom")` as the second argument, we'll still get a result since we never need to force the value inside of the `Maybe`:

```
λ lazyGuard (Just 20) (Just (error "boom"))
"Greater than 10"
```

Pattern Guards and case Expressions

All the examples we've looked at so far have involved function guards, but pattern guards can also be used with case expressions. Let's look at an example:

```
src/HaskellBrainTeasers/PlayingWithMatches/Examples.hs
describePair :: Maybe Int -> Maybe Int -> String
describePair maybeX maybeY =
  | Just x <- maybeX, Just y <- maybeY =
    case compare x y of
      EQ -> "equal"
      GT | even x, even y -> "descending even"
      LT | odd x, odd y -> "ascending odd"
      _ -> "changing"
  | otherwise = "incomparable"
```

This function shows off a common pattern where we use a case expression to pattern match on some larger structure, then use guards to refine the match. You'll notice that the guards we're using mean that matching on `GT` or `LT` is partial. This behaves as you'd expect, with branches getting evaluated from top to bottom lazily. If we match on `GT` but have odd values, the overall pattern fails and we'll fall through to our wildcard case at the bottom. GHC's exhaustiveness checker is typically able to warn you when case patterns and guards leave you with incomplete or unreachable code.

Further Reading

Pattern Guards in the GHC User's Guide

https://downloads.haskell.org/ghc/latest/docs/users_guide/xts/pattern_guards.html#pattern-guards

Pattern guard on the Haskell wiki

https://wiki.haskell.org/Pattern_guard

Pattern Guards and Transformational Patterns

<https://www.microsoft.com/en-us/research/wp-content/uploads/2016/07/pat.pdf>

Pythagorean triple on Wikipedia

https://en.wikipedia.org/wiki/Pythagorean_triple

Puzzle 9

Many Roads, One Destination

```
src/HaskellBrainTeasers/Bismuth.hs
{-# LANGUAGE OrPatterns #-}
module HaskellBrainTeasers.Bismuth where

data Color = Black|White|Gray|Red|Green|Blue|Yellow|Purple|Orange
  deriving Show

isAchromatic :: Color -> Bool
isAchromatic (Black;White;Gray) = True
isAchromatic (Red;Green;Blue;Yellow;Purple;Orange) = False

main :: IO ()
main = print (isAchromatic Red) >> print (isAchromatic Gray)
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

The program will output
False
True

Discussion

This puzzle demonstrates a newer GHC language extension called *or-patterns*. Or-patterns let you combine several otherwise identical pattern matches into a single larger pattern. Without or-patterns, we would have needed to choose between either a much more verbose implementation of our puzzle or one that risked misbehaving in the future.

As an example, we could have written a version of `isAchromatic` that explicitly pattern matched on every color. This would have turned our two pattern matches into nine pattern matches. More likely, we would have written our function using a wildcard to save some typing:

```
src/HaskellBrainTeasers/Bismuth.hs
isAchromaticWildcard :: Color -> Bool
isAchromaticWildcard Black = True
isAchromaticWildcard White = True
isAchromaticWildcard Gray = True
isAchromaticWildcard _ = False
```

This implementation of our function works well today, but it might not always behave as expected in the future. Imagine that we add two new achromatic colors: `LightGray` and `DarkGray`. If we were diligent about explicitly pattern matching on every color, the compiler would tell us that we now have an incomplete pattern and point us to the offending function. This is useful, but it means our already long function gets even longer. If, on the other hand, we took a shortcut and used a wildcard, then our two new colors would hit the wildcard. The compiler wouldn't warn us, and we'd suddenly find ourselves claiming that neither `LightGray` nor `DarkGray` is achromatic.

As you can see in the puzzle, or-patterns can help us get the best of both worlds by making it easier to explicitly enumerate every case with less verbosity.

Limitations

Although or-patterns are useful and can let us avoid wildcards or partial patterns, they do have some limitations that we need to work around in order to use them effectively. Most notably, or-patterns do not support binding variables or constraints.

Let's start by taking a look at variable binding. In a normal pattern match, we can bind a variable to part of the pattern. For example, working with `Either` values, you can normally bind a variable to the `Left` or `Right` value. Imagine we want to write a function that takes a pair of `Either` values and checks to see if they're "mirrors" of one another:

```
src/HaskellBrainTeasers/Bismuth/MirrorTest.hs
isMirrored :: (Either Int Int, Either Int Int) -> Bool
isMirrored (Left x, Right y) = x == y
isMirrored (Right x, Left y) = x == y
isMirrored ((Left _, Left _); (Right _, Right _)) = False
```

In this example, `x` and `y` are variables we're binding to the `Left` and `Right` values inside our patterns. We can do this in the example because the first two patterns we're using are not or-patterns. In the final pattern, we're using an or-pattern to match the case when we get two `Left` or two `Right` values, but we're not binding a variable in that case, so this also works as expected.

Looking at the following example, you might notice that our first two patterns both implement the same logic and consider combining them into an or-pattern as well:

```
src/HaskellBrainTeasers/Bismuth/MirrorTest.hs
isMirrored :: (Either Int Int, Either Int Int) -> Bool
isMirrored ((Left x, Right y); (Right x, Left y)) = x == y
isMirrored ((Left {}, Left {}); (Right {}, Right {})) = False
```

Unfortunately, this is where we run into the limitation on binding variables in or-patterns. In this example, we've combined our first two patterns into a single or-pattern, including attempting to bind `x` and `y`. Since we need to bind variables in our pattern, we'll have to go back to our earlier example and reserve the or-pattern only for the fall-through case.

Variable binding is something visible that we do intentionally when we need a value. Although we may wish to bind variables in an or-pattern, it's clear when we're trying to do so and easy enough to revert back to matching patterns individually. The lack of support for binding constraints in or-patterns is a little trickier since this is something the compiler often does for us implicitly. Let's look at an example of some code we might write that works as expected without or-patterns:

```
src/HaskellBrainTeasers/Bismuth/ValueTypes.hs
{-# LANGUAGE OrPatterns #-}
{-# LANGUAGE GADTs #-}
module HaskellBrainTeasers.Bismuth.ValueTypes where
import Data.Word

newtype Each = Each Word32 deriving (Num, Show)
```

```

newtype Pounds = Pounds Double deriving (Num, Fractional, Show)

data Inventory a where
  Toasters :: Inventory Each
  Iguanas  :: Inventory Each
  EngineGreas :: Inventory Pounds
  Antimatter :: Inventory Pounds

inventory :: Inventory a -> a
inventory Toasters = 0
inventory Iguanas  = 0
inventory EngineGreas = 0
inventory Antimatter = 0

```

Here we have an `Inventory` type that represents different items we might carry in a shop. We can define our store's inventory at a moment in time by providing a function from the inventory item name to the amount that we have in stock. Our current inventory says that we're out of stock on all our items.

Knowing about or-patterns, you might notice that since our inventory items are symbolic and we don't need to bind a variable, this could be a good opportunity to use or-patterns. After all, this is exactly the kind of problem where we might want to avoid a wildcard pattern that might not be correct for new items we add to our inventory. Unfortunately, putting everything into a single or-pattern doesn't work:

```
src/HaskellBrainTeasers/Bismuth/ValueTypes.hs
```

```

inventory :: Inventory a -> a
inventory (Toasters; Iguanas; EngineGreas; Antimatter) = 0

```

The problem is that the type of value we return depends on the specific constructor we've matched on. We count our inventory by `Each` `Toasters` or `Iguanas` but by how many `Pounds` we have in stock of `EngineGreas` or `Antimatter`. Your first thought might be to use two or-patterns, one for `Each` items and one for `Pounds` items:

```
src/HaskellBrainTeasers/Bismuth/ValueTypes.hs
```

```

inventory :: Inventory a -> a
inventory (Toasters; Iguanas) = 0
inventory (EngineGreas; Antimatter) = 0

```

Disappointingly, this still won't work. Normally, when pattern match on a generalized algebraic data type (GADT), the compiler will generate a constraint that tells us what type we're dealing with. In our working example, when we pattern match on `Toasters`, we introduce a constraint that says the type of `a` has to be `Each`. Since or-patterns don't bind constraints, we lose this information and the compiler can't be sure that what we're returning is correct.

Or-patterns are a new feature as of GHC 9.12, and they may become more capable in future versions. Today, they can be very useful for functions that need to match on simple sum types with many common code paths, but their limitations mean that they aren't always a viable choice even when they seem potentially useful.

Further Reading

Or-patterns in the GHC User's Guide

https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/or_patterns.html

GHC Proposal 0522: Or-patterns

https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/or_patterns.html

A View of the Future

src/HaskellBrainTeasers/FindMap.hs

```
{-# LANGUAGE ViewPatterns #-}
module HaskellBrainTeasers.FindMap where
import Data.List

findMap :: (a -> Maybe b) -> [a] -> Maybe b
findMap p (uncons -> Just (p -> Just b, _rest)) = Just b
findMap p (uncons -> Just (_, rest)) = findMap p rest
findMap _ (uncons -> Nothing) = Nothing

main :: IO ()
main = print $ findMap toEven [1,3,5,6,7,9]
  where
    toEven :: Int -> Maybe Int
    toEven n = if even n then Just n else Nothing
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

The program will output
Just 6

Discussion

This puzzle introduces yet another extension to pattern matching in Haskell: view patterns. View patterns give us a convenient way to handle the common case where we want to pattern match on the result of applying a function to a value. View patterns let us call a function on a value and pattern match on the result without having to name the initial value. At a high level, it lets us replace something like

```
f a =
  case (view a) of
    Pattern1 -> result
    _ -> otherResult
```

with a shorter alternative:

```
f (view -> Pattern1) = result
f _ = otherResult
```

Like other puzzles in this book that show off extensions to pattern matching, view patterns are not strictly necessary. You can replace code using view patterns with regular pattern matches, case expressions, and pattern guards. For example, here's how we could have written our puzzle using pattern guards and case expressions:

```
src/HaskellBrainTeasers/FindMap.hs
{-# LANGUAGE ViewPatterns #-}
module HaskellBrainTeasers.FindMap where
import Data.List

findMap :: (a -> Maybe b) -> [a] -> Maybe b
findMap p (uncons -> Just (p -> Just b, _rest)) = Just b
findMap p (uncons -> Just (_, rest)) = findMap p rest
findMap _ (uncons -> Nothing) = Nothing

main :: IO ()
main = print $ findMap toEven [1,3,5,6,7,9]
  where
    toEven :: Int -> Maybe Int
    toEven n = if even n then Just n else Nothing
```

At their best, view patterns can give us a quick and easy way to wrap function calls or perform some lightweight computation on a piece of data. For example,

we can use view patterns to easily write a helper that calls a function on an element of a Map if it exists:

```
src/HaskellBrainTeasers/FindMap/WithElem.hs
```

```
withElem :: Ord k => (v -> r) -> k -> Map k v -> Maybe r
withElem f key (Map.lookup key -> Just val) = Just (f val)
withElem _f _key _val = Nothing
```

At their worst, deeply nested view patterns can become a strange way of pipelining computations on nested data structures. For example, imagine we wanted a function to check if we know someone's spouse's grandparent's middle name:

```
src/HaskellBrainTeasers/FindMap/WithElem.hs
```

```
data Person = Person
  { firstName :: String
  , middleName :: Maybe String
  , lastName :: String
  , spouses :: [Person]
  , parents :: [Person]
  }

spousesGrandparentsNameMatches :: String -> Person -> Bool
spousesGrandparentsNameMatches
  name
  (spouses ->
    concatMap parents ->
    concatMap parents ->
    map middleName ->
    grandparentMiddleNames) =
  Just name `elem` grandparentMiddleNames
```

Overall view patterns are a matter of taste and can, when used judiciously, help you write small helper functions quickly and easily.

Further Reading

View patterns in the GHC User's Guide

https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/view_patterns.html#view-patterns

View patterns on the GHC wiki

<https://gitlab.haskell.org/ghc/ghc/-/wikis/view-patterns>

FQDN - Fully Qualified do Notation

```
src/HaskellBrainTeasers/AQualifiedSuccess.hs
{-# LANGUAGE QualifiedDo #-}
module HaskellBrainTeasers.AQualifiedSuccess where

(>>=) :: Either a b -> (a -> Either c b) -> Either c b
Left a >>= f = f a
Right b >>= _ = Right b

return :: a -> Either a b
return = Left

main :: IO ()
main = print unqualifiedHello >> print qualifiedHello
  where
    unqualifiedHello :: Either String String
    unqualifiedHello = do
      hello <- Left "hello"
      world <- Left "world"
      pure $ hello <> " " <> world
    qualifiedHello :: Either String String
    qualifiedHello = HaskellBrainTeasers.AQualifiedSuccess.do
      hello <- Left "hello"
      world <- Left "world"
      pure $ hello <> " " <> world
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

The program will output
 Left "hello"
 Right "hello world"

Discussion

This puzzle shows how the `QualifiedDo` language extension allows us to write Haskell programs in a way we wouldn't otherwise be able to—as well as how those programs can sometimes violate our intuition for how our programs should behave.

do Notation

Haskell's `do` notation gives us a convenient shorthand for writing monadic code. When we use it with types such as `IO`, it lets us write pure functional code in a way that feels idiomatically procedural. With types such as `Maybe` and `Either`, it lets us easily model otherwise pure computations that might fail. Fundamentally, though, ordinary `do` notation is just syntactic sugar for monadic operations such as `bind (>>=)` and `return`. For example, we can compare a function written in `do` notation with its desugared counterpart:

[src/HaskellBrainTeasers/AQualifiedSuccess/DoExample.hs](#)

```
pickThreeWithDo :: [a] -> Maybe (a,a,a)
pickThreeWithDo xs = do
  (x, xs') <- uncons xs
  (x', xs'') <- uncons xs'
  (x'', _) <- uncons xs''
  return (x, x', x'')

pickThreeWithBind :: [a] -> Maybe (a,a,a)
pickThreeWithBind xs =
  uncons xs >>= \ (x, xs') ->
  uncons xs' >>= \ (x', xs'') ->
  uncons xs'' >>= \ (x'', _) ->
  return (x, x', x'')
```

Without any language extensions enabled, all `do` notation in Haskell gets desugared into `(>>=)` and `return` just like in our example. The specific implementations of `(>>=)` and `return` that get called come from the `Monad` instance of whatever value the `do` block returns.

Applicative do

The fact that you need a `Monad` instance to use `do` notation isn't typically a problem, but after `Applicative` was standardized and made a requirement of `Monad`, people started to realize that there are some cases where `do` notation

would be useful even for types that are `Applicative` but can't or don't have `Monad` instances. `ApplicativeDo` was added in GHC 8. With this extension enabled, you can use `do` notation and the compiler will attempt to desugar your code into calls to `(<$>)` from `Functor` and `(<*>)` from `Applicative`. If the code you write in a `do` block has data dependencies that require it, the compiler will still fall back to using `(>=>)` when `ApplicativeDo` is enabled. In those cases, you'll get an error if you haven't provided a `Monad` instance.

Let's look at an example of a function that uses `do` notation and how it might be desugared when we've enabled `ApplicativeDo`:

```
src/HaskellBrainTeasers/AQualifiedSuccess/ApplicativeDo.hs
combineThreeDo ::
  Maybe a -> Maybe a -> Maybe a -> Maybe (a,a,a)
combineThreeDo a b c = do
  a' <- a
  b' <- b
  c' <- c
  pure (a', b', c')

combineThreeAp ::
  Maybe a -> Maybe a -> Maybe a -> Maybe (a,a,a)
combineThreeAp a b c = (,,) <$> a <*> b <*> c
```

Using `ApplicativeDo` lets us break free of the need for a `Monad` instance if we want to use `do` notation, but it's still somewhat restrictive. Both `Applicative` and `Monad` are defined for types of the kind `(Type -> Type)`. In other words, they need to be defined for a type with one type variable in the right-hand position that can vary as we call functions such as `<$>` or `(>=>)`.

For types such as lists, `Maybe`, and `IO` that only have a single type argument, this requirement isn't restrictive at all. For types such as `Either`, the requirement is a little more restrictive because we have to "fix" the first argument. That's why the convention for `Either` is to use `Left` values for error handling and `Right` for the result of the computation.

Qualified do

The `QualifiedDo` extension goes a step further than `ApplicativeDo` and removes any type class constraints on our ability to use `do` notation. Instead of looking for definitions of `(>=>)` or return from a type class instance, `QualifiedDo` allows us to define these functions as ordinary Haskell functions with types that don't necessarily need to match the types in the `Monad` or `Applicative` type classes.

Removing this restriction can be particularly powerful when we need more power at the type level to express what it means to sequence operations. In trivial cases, such as our puzzle, this could be as simple as varying the first

type argument rather than a later type argument. We can also use `QualifiedDo` for much more powerful APIs. For example, in just a few lines of code, we can create a Reader environment that tracks each access at the type level:

```
src/HaskellBrainTeasers/AQualifiedSuccess/Audit.hs
{-# LANGUAGE AllowAmbiguousTypes #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE RequiredTypeArguments #-}
module HaskellBrainTeasers.AQualifiedSuccess.Audit where
import GHC.TypeLits
import Data.Map.Strict qualified as Map
import Data.Proxy

type Env = Map.Map String Int

type family (xs :: [a]) :++: (ys :: [a]) :: [a] where
  '[] :++: ys = ys
  (x ': xs) :++: ys = x ': (xs :++: ys)

newtype Reader (keys :: [Symbol]) env a where
  Reader :: {runReader :: env -> Maybe a} -> Reader keys env a

lookup ::
  forall key -> KnownSymbol key => Reader '[key] Env Int
lookup key = Reader $ Map.lookup (symbolVal $ Proxy @key)

return :: a -> Reader '[] env a
return = Reader . const . Just

(>>=) ::
  Reader j e a -> (a -> Reader k e b) -> Reader (j:++:k) e b
(Reader f) >>= g = Reader $ \env -> do
  result <- f env
  runReader (g result) env

class ShowSyms (keys :: [Symbol]) where symbols :: [String]
instance ShowSyms '[] where symbols = []
instance (KnownSymbol x, ShowSyms xs) => ShowSyms (x:xs) where
  symbols = symbolVal (Proxy @x) : symbols @xs

audit :: forall keys env a. ShowSyms keys =>
  Reader keys env a -> env -> (Maybe a, [String])
audit ar env = (runReader ar env, symbols @keys)
```

As you can see, this module is short but conceptually dense. The core of the module is a Reader that lets us access keys from an environment and tracks which keys we've accessed at the type level. We can read a key with the `lookup` function and get a Reader that tracks the lookup:

```
λ :type lookup "at the sky"
lookup "at the sky"
  :: KnownSymbol "at the sky" =>
    Reader '["at the sky"] Env (Maybe Int)
```

The `audit` function runs a `Reader` computation and returns a value. Thanks to the recursive `ShowSyms` type class instance we're also able to get a list of `String` values from the type-level list of keys that the `Reader` will access.

```
λ audit (lookup "at the sky") $ Map.fromList [("at the sky", 0)]
(Just 0, ["at the sky"])
```

In addition to `audit`, we have two other ordinary top-level functions: `return` and `(>>=)`. These two functions share names with the functions from the `Monad` type class, and their behavior is similar. `return`, for example, will give us a new `Reader` that doesn't lookup any values:

```
λ import HaskellBrainTeasers.AQualifiedSuccess.Audit as Audit
λ :type Audit.return (10 :: Int)
Audit.return (10 :: Int) :: Reader '[] env Int
```

The `(>>=)` operator we've defined lets us combine `Reader` computations in a way similar to how we'd use the monadic `(>>=)` operator:

```
bindReader :: Reader '["zero", "one"] (Map.Map String Int) Int
bindReader =
  Audit.lookup "zero" Audit.>>= \zero ->
  Audit.lookup "one" Audit.>>= \one ->
  Audit.return (zero + one)
```

Even though `return` and `(>>=)` act like their counterparts from `Monad`, we couldn't use them to define a `Monad` instance. In a `Monad`, only the last type argument is allowed to change, but our `Reader` accumulates new keys each time we call `lookup`. It turns out that `Reader` is an example of a *graded monad*, which is one of the motivating reasons for the `QualifiedDo` extension.

In our example, `QualifiedDo` gives us two significant advantages: It lets us use `do` notation with types that behave monadically but can't provide a proper `Monad` instance, and it allows us to encapsulate more advanced Haskell code behind an accessible internal API. Let's look at an example:

```
src/HaskellBrainTeasers/AQualifiedSuccess/UseAudit.hs
{-# LANGUAGE QualifiedDo #-}
module HaskellBrainTeasers.AQualifiedSuccess.UseAudit where
import Data.Map.Strict qualified as Map
import HaskellBrainTeasers.AQualifiedSuccess.Audit as Audit

runWithAuditing ::
  (Show a, Audit.ShowSyms keys) =>
  Audit.Env -> Audit.Reader keys Audit.Env a -> IO ()
runWithAuditing env reader =
  let (result, auditTrail) = Audit.audit reader env
  in print result >> putStrLn ("keys: " <> show auditTrail)

main :: IO ()
```

```
main = runWithAuditing env $ Audit.do
  one <- Audit.lookup "one"
  two <- Audit.lookup "two"
  Audit.return $ one + two
  where env = Map.fromList [("one", 1), ("two", 2), ("three", 3)]
```

In this example we're able to write code using `Audit.do` in the same way we'd use normal `do` notation if we had a `Monad` instance. Although the underlying implementation makes use of more advanced features, the interface we've provided is accessible even to developers with less Haskell experience.

Further Reading

- Qualified `do` notation in the GHC User's Guide²
- Applicative `do` notation in the GHC User's Guide³
- The Reader Monad⁴
- Unifying Graded and Parameterised Monads⁵

2. https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/qualified_do.html#extension-QualifiedDo

3. https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/applicative_do.html

4. <https://hackage.haskell.org/package/mtl-2.3.1/docs/Control-Monad-Reader.html>

5. <https://arxiv.org/abs/2001.10274>

Part III

Type Classes

It's time to get schooled in the weird, wonderful, and sometimes unintuitive world of type classes. In this series of puzzles, you'll need to think about how type classes are defined and how to create instances of them, and you'll need to consider just how far the laws that govern some of the common type classes in the standard library can take you. As you work through these puzzles, remember to check Hackage for the definitions and laws that govern these classes. If you find that a puzzle is too difficult, skip it and try again later or read the detailed explanation and try to come up with your own example that demonstrates the behavior.

Traversing Tuples

```
src/HaskellBrainTeasers/TheWholeWideWorld.hs
```

```
module HaskellBrainTeasers.TheWholeWideWorld where
import Data.Foldable (traverse_)

main :: IO ()
main = traverse_ putStrLn $ do
  greeting <- ("ToTheWholeWide", "World")
  say "Hello" greeting
  where
    say a b = (a<>) <$> (b,b)
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

The program will output
HelloWorld

Discussion

This puzzle demonstrates the Functor, Applicative, Monad, and Foldable instances of tuples. These are all lawful instances supplied by base, but they're also common sources of confusion. Even experienced Haskell developers sometimes forget these instances exist, and most people find the instances confusing the first time they encounter them. In fact, most people's first response to seeing them used is, "Thanks, I hate it."

Surprisingly, it turns out that while they might be unintuitive at first, they are also the simplest useful instances possible. To understand why, let's walk through them one by one and look at how each instance arises naturally.

Functor

The first time you use the Functor instance, it's common to find yourself thinking, "What in the `fmap` is happening here?"

```
λ show <$> (1,10)
(1, "10")

λ (+1) <$> (2,3)
(2,4)
```

From these examples, it looks like `fmap` is always applying the function to the second argument of the tuple. It turns out that's exactly what's happening. Let's create our own version of a tuple and write a Functor instance to take a closer look.

```
newtype Tuple a b = Tuple (a,b)

instance Functor (Tuple a) where
  fmap f (Tuple (a, b)) = Tuple (a, f b)
```

It's fairly easy to understand *what* is happening here, but at first glance the choice to apply the function to the second argument seems weird and unintuitive. Let's consider what more intuitive instances might look like.

The first and most obvious idea would be to treat tuples like lists and apply the function to each argument. But that idea falls apart pretty quickly under scrutiny. Unlike lists, where every element has to have the same type, each element of a tuple can be different. We might call `fmap` with a function that happens to be polymorphic enough to handle both types in a tuple, but there's

no guarantee that it will be. The easiest and most general solution is to pick one of the two types and accept a function that operates on that type.

This naturally leads to the next most obvious idea: If `fmap` can only apply to one argument, why not make it apply to the first argument rather than the second? Haskell is written left to right, so it feels intuitive that the first element we encounter when reading or writing tuples is more important and therefore should be the element that gets applied when we call `fmap`.

It turns out that the intuition that we should be applying a function to the “first” element isn’t entirely wrong, but we’re thinking about the problem at the term level when we should be thinking about it at the type level, or more accurately, at the *kind* level.

The Functor, Applicative, and Monad type classes are all defined for types that take a single argument. We create instances for `Maybe`, not `Maybe Int`, or for `IO`, not `IO BirdPicture`. Intuitively, this makes sense—the whole point of `fmap` is that it applies a function to the values in the structure without changing the structure itself. If we put on our top hat and monocle and speak in fancy Haskell terms, we would say that these type classes are defined for types of *kind* `Type -> Type`.

At this point, an astute reader will smugly point out that this is completely wrong. After all, we have Functor instances for `Either` and for tuples—in fact, there’s an entire *Haskell Brain Teasers* puzzle about the instance for tuples!

The trick is that when we define these instances for a type with more than one parameter, we need to fix all but the last argument. We don’t write `instance Functor Either` or `instance Functor Tuple`; instead, we write `instance Functor (Either a)` or `instance Functor (Tuple a)`. The type variables here are polymorphic, but they still need to remain fixed. Whenever we call `fmap` with a type such as `Either a b` or `Tuple a b`, both the argument and the return value need to keep the same value for `a`. This is easy to see if we look at the type of `fmap` and substitute different values for `f`:

```
fmap :: Functor f => (a -> b) -> f a -> f b
fmap :: (b -> c) -> Either a b -> Either a c
fmap :: (b -> c) -> Tuple a b -> Tuple a c
```

When you understand how kinds force us to fix some parameters of a type in an instance, it’s easy to see that applying the function to the second argument of a tuple isn’t weird—it’s simply choosing the first (and only) argument available to us in the instance.

Applicative and Monad

Like Functor, both Applicative and Monad are defined for types of the kind `Type -> Type`, and both require that we fix the first argument. But this turns out to be a big problem for Applicative. To understand why, let's look at the definition of `pure` as it appears in the Applicative type class, then at how it's implemented for `Either` and at how we might implement it for `Tuple`.

```
-- The general definition for all Applicatives
pure :: Applicative f => a -> f a

instance Applicative (Either a) where
  pure b = Right b

instance Applicative (Tuple a) where
  pure b = (error "boom!", b)
```

The instances for both `(Either a)` and `(Tuple a)` have to deal with the fact that their instances include an extra type parameter that isn't accounted for when implementing `pure`. This isn't a problem for `Either`, because the type `a` exists but we never have to summon a value for it. The `Right` constructor only needs a value of type `b`.

Our `Tuple` instance isn't so lucky. We get a value of type `b`, but somehow we still need to summon a value of type `a`—but `a` is a fully polymorphic value. We don't know anything about it, so we can't create a value or do anything else with it. Grab your top hat and monocle and say *parametricity*.

Thankfully, there's an escape hatch. If we add a constraint to the instance, then we can learn enough about `a` to create a useful value. The `Monoid` class gives us a way to create an “empty” value named `mempty`. This gets us the first part of our real instance:

```
instance Monoid a => Applicative (Tuple a) where
  pure b = (mempty, b)
```

The choice to use `Monoid` is serendipitous because Applicative and Monad do more than just allow us to create new values. They allow us to *combine* structures. The `(<*>)` allows us to combine structures in parallel, while `(>=>=)` allows us to combine nested structures. If you have your top hat and monocle handy, this would be a good time to say, “The Applicative and Monad laws say they must combine monoidally.”

Structure can show up in a number of different ways. The structure of a list is the number and order of elements. In `Maybe`, structure is the presence or absence of a value. And for `Either`, structure describes whether we have a `Left` or `Right` value. You can think of structure as metadata about a data structure.

If the value of a tuple is the value of its second element (as far as Functor, Applicative, and Monad are concerned), then the structure of a tuple is the value of its first element. Since that first element already has to be a Monoid, we can use ($\lt\>$) to combine our structures. This leaves us with fairly obvious instances:

```
src/HaskellBrainTeasers/TheWholeWideWorld/Instances.hs
instance Monoid a => Applicative (Tuple a) where
  pure a = Tuple (mempty, a)
  (Tuple (a, f)) <*> (Tuple (b, x)) = Tuple (a <> b, f x)
instance Monoid a => Monad (Tuple a) where
  return = pure
  (Tuple (a, x)) >>= f =
    let (Tuple (b, y)) = f x in Tuple (a <> b, y)
```

Writing a Conclusion

Understanding how Haskell pushes us toward these specific instances might make them seem less capricious, but you may still wonder if they're useful. It turns out that with these instances, tuples are precisely Writer, a computation that allows us to write information, such as log files.

Further Reading

The Functor instance for tuples : <https://hackage.haskell.org/package/ghc-internal-9.1201.0/docs/src/GHC.Internal.Base.html#line-1552>

The Monad instance for tuples : <https://hackage.haskell.org/package/ghc-internal-9.1201.0/docs/src/GHC.Internal.Base.html#line-908>

The lazy writer Monad : <https://hackage.haskell.org/package/mtl-2.3.1/docs/Control-Monad-Writer-Lazy.html>

A Slightly Off-White Hat

```
src/HaskellBrainTeasers/MadHacks.hs
module HaskellBrainTeasers.MadHacks where
data Auth a = Unauthenticated | Authenticated a
    deriving Show
instance Functor Auth where
    fmap h4x0r l33t = l33t >=> return . h4x0r
instance Applicative Auth where
    pure = return
    l33t <*> h4x0r = do
        el33t <- l33t
        h4x0r >=> return . el33t
instance Monad Auth where
    return l33t = Authenticated l33t
    (>=>) l33t h4x0r
        | Authenticated pwnd <- l33t = h4x0r pwnd
        | otherwise = Unauthenticated

main :: IO ()
main = print $
    ("access granted to " <>) <$> Authenticated "User1"
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

The program will output
Authenticated "access granted to User1"

Discussion

In this puzzle, we're using our elite hacking skills to get access to a user's account, but we're not just going through the back door to get user access. We've also gone completely backwards through the usual Functor, Applicative, and Monad type class hierarchy. Instead of starting with a simple Functor instance and gradually adding more power with Applicative and then Monad, we've implemented Monad and then implemented our other two instances in terms of ($\gg=$) and return.

Although it's surprising the first time you see type class instances that seemingly subvert the type class hierarchy, it turns out to be both a natural consequence of how type classes are implemented and a technique with practical applications. As a concrete example, Monad has only been a subclass of Applicative and Functor since GHC version 7.10. Today, the hierarchy is familiar.

[src/HaskellBrainTeasers/MadHacks/ClassDefinitions.hs](#)

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
class Functor f => Applicative f where
  pure :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b
class Applicative m => Monad m where
  return :: a -> m a
  (>>=) :: m a -> (a -> m b) -> m b
```

Until the Applicative $\Rightarrow$ Monad proposal (AMP), there was nothing that required you to implement the full hierarchy, and in practice many developers skipped Applicative entirely since it was usually unnecessary and not well understood. Anyone who has ever had to deal with breaking changes in software can easily imagine the potential for chaos when introducing something as significant as changes to Monad. The rollout of AMP wasn't without some chaos, but the process was made significantly easier since most maintainers were able to define Applicative instances in terms of the Monad instances they already had. Even today, you may run across code with definitions of ($\lt\ast\gt$) that rely on a type's Monad instance. The general process of implementing superclasses in terms of subclass instances remains a viable approach both for incrementally rolling out type class hierarchy changes and as a way of reusing complex or highly optimized subclass instances.

Hacking the Hierarchy

The first time you see a type class instance implemented in terms of its subclass, it can feel a little magical. Especially where subclasses are simpler to implement, it can feel like a way to avoid figuring out how to write an instance yourself. Unfortunately, this approach can also lead to some common failure cases. The three most common problems you'll run into are circular references in instances, missing subclass definitions, and missing type class constraints.

Circular instance definitions happen when a superclass refers to functions defined by a subclass and the subclass instance refers back to those same functions in the superclass. A trivial example is implementing `pure` and `return` in terms of one another. When this happens, the compiler can't tell the difference between an accidental loop and intentional cyclic references, and so you'll see code that compiles as expected but hangs forever at runtime.

In theory, this can happen anywhere, not just in type class instances. Every Haskeller has a story of accidentally creating a fixed point, but it's particular insidious with type class instances because they're one of the few ways you can create circular references across module, or even package, boundaries.

Thankfully, the compiler can catch the other two common causes of errors when you write an instance that depends on a subclass: missing subclass instances and missing constraints. These problems often show up together, so let's look at a single example that will let us illustrate them both:

```
src/HaskellBrainTeasers/MadHacks/WrapperInstances.hs
newtype Wrapper m a = Wrapper { unwrap :: m a }
instance Functor m => Functor (Wrapper m) where
  fmap f (Wrapper ma) = Wrapper (f <$> ma)
instance Applicative m => Applicative (Wrapper m) where
  pure a = Wrapper (pure a)
  (Wrapper mf) <*> (Wrapper ma) = Wrapper (mf <*> ma)
instance Monad m => Monad (Wrapper m) where
  return = Wrapper . return
  (Wrapper ma) >>= f = Wrapper $ ma >>= unwrap . f
```

Our `Wrapper` instances provide perfectly reasonable instances for our type classes. But then we decide to try rewriting our `Functor` instance in terms of `Monad`:

```
instance Functor m => Functor (Wrapper m) where
  fmap f ma = ma >>= return . f
```

Unfortunately, if we try to build this, we'll quickly get an error:

```
HaskellBrainTeasers/MadHacks/WrapperInstances.hs:23:64-66: error: [GHC-39999]
- Could not deduce 'Monad m' arising from a use of '>>='
- from the context: Functor m
```

The problem with our new Functor (Wrapper m) instance is that we're trying to use the Monad (Wrapper m) instance, but that instance only exists if m also has a Monad instance. As written, we have no guarantee that m is a Monad, only that it has a Functor instance. This is an easy fix in theory. We just need to replace the (Functor m) constraint with a (Monad m) constraint:

```
instance Monad m => Functor (Wrapper m) where
  fmap f ma = ma >>= return . f
```

This change fixes our immediate problem but leaves us with a new problem: Our Applicative (Wrapper m) instance is now missing a (Monad m) constraint. We can add one and our program will compile, but it's easy to misunderstand why the extra constraint is needed. It's common to think of constraints as “pulling in a dependency,” and that made sense for our Functor (Wrapper m) instance since we changed it to use the Monad (Wrapper m) instance, but we haven't changed the code in our Applicative (Wrapper m) at all.

It's easier to spot the problem if we change how we see constraints. Instead of “pulling in capabilities,” we can see constraints as limiting which instances we're creating. For an example, let's return to our Functor (Wrapper m) instance:

```
instance Monad m => Functor (Wrapper m) where
  fmap f ma = ma >>= return . f
```

We can read this as ‘*Wrapper m*’ has an instance of ‘*Functor*’ if, and only if, ‘*m*’ has an instance of ‘*Monad*’. This is a much more literal interpretation of the word *constraint*: It constrains the instances that we're generating.

The same idea applies to constraints on type classes themselves. Take the following type class with a constraint:

```
class Functor m => Applicative m where
```

We can read this as saying, “Applicative is a subclass of Functor,” or if we prefer mathy lingo, we can say “Applicative m entails Functor m.” If we want to apply our same “constraints constrain what exists” lens, we might say something like, “The class of types Applicative m is defined only for types where there is an instance of Functor m.”

When we apply this view of constraints and instances, we can immediately see the problem. We're trying to claim all of the following things are true:

- A Functor (Wrapper m) instance exists anytime there is a Monad m instance
- An Applicative (Wrapper m) can only exist if there's a Functor (Wrapper m) instance
- An Applicative (Wrapper m) instance exists any time there's an Applicative m instance

This falls apart because not every `Applicative m` is a `Monad m`. Our `Applicative` instance claims to exist whenever `Applicative m` is defined, but if there isn't also a `Monad m` instance, then we haven't defined the `Functor (Wrapper m)` instance that must be defined for the `Applicative (Wrapper m)` instance to exist. The generalization of this problem tells us that a subclass must always be at least as constrained as all its superclasses.

Unfortunately, even after we've cleaned up our constraints and gotten the module to build, we can still run into problems if we were using `Wrapper` with a type that doesn't have a `Monad` instance. Take the following example:

```
incrementWrappedMap ::
  Wrapper (Map String) Int -> Wrapper (Map String) Int
incrementWrappedMap m = (+1) <$> m
```

This function would have worked as expected before we implemented `Functor (Wrapper m)` in terms of `Monad (Wrapper m)`, but now that we've refactored our code, the function will fail to compile since there's no `Monad` instance for `Map`.

Further Reading

`Functor`, `Applicative`, and `Monad` in base : <https://hackage.haskell.org/package/ghc-internal-9.1201.0/docs/src/GHC.Internal.Base.html#Functor>

The `Functor-Applicative-Monad` Proposal < : https://wiki.haskell.org/Functor-Applicative-Monad_Proposal

Haskell, IO to Core : <https://serokell.io/blog/haskell-to-core>

If At First You Don't Succeed, Trie, Trie Again

```
src/HaskellBrainTeasers/WhatTheDuck.hs
module HaskellBrainTeasers.WhatTheDuck where
import Control.Monad
import Data.List

dict :: [(Char, Int, String)]
dict = [('h',0,"eo"), ('e',1,"l$"), ('l',2,"lp")
        ,('l',3,"o"), ('p',3,"$"), ('o',4,"$")]

autoComplete :: String -> [String]
autoComplete [] = []
autoComplete prefix@(firstChar:_) =
    filter (isPrefixOf prefix) $ step firstChar 0
    where
        step curChar curIdx = do
            (entryChar,entryIdx,entryOut) <- dict
            guard $ entryChar == curChar && entryIdx == curIdx
            nextChar <- entryOut
            result <- if nextChar == '$'
                then pure ""
                else step nextChar (curIdx + 1)
            pure $ curChar : result

main :: IO ()
main = print $ autoComplete "hel"
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

```
The program will output
["hello", "help"]
```

Discussion

No, you're not quacking up. We're using `do` notation with lists because this is Haskell, and if we can, we'll put a `Monad` on it. In this case, we're implementing `autocomplete`, because writing a twelve line monadic traversal of a tree of possible word completions is more satisfying than typing out a pair of words like a chump.

The list monad is one of Haskell's great ironies: ubiquitous in blog posts and monad tutorials but rarely used and often cursed in real production code. As our puzzle demonstrates, though, in the right circumstances, it can be a practical and useful technique for small search problems and dealing with nondeterminism.

Let's start by looking at how the `Functor`, `Applicative`, and `Monad` instances for lists work, then we'll move on to looking at two distinct ways to think about these instances and how they let us solve different problems.

Functor, Applicative, and Monad for Lists

The `Functor` instance for lists is interesting only in how its incredible mundanity stands in contrast to more surprising instances:

```
src/HaskellBrainTeasers/WhatTheDuck/Instances.hs
newtype List a = List {toList :: [a]}
instance Functor List where
    fmap f (List xs) = List (map f xs)
```

The standard definition of `fmap` for lists is just `map`, and it behaves exactly as you'd expect. In our example, we've wrapped standard lists in a custom `List` type to make it easier to demonstrate how the instances work. Let's move on to something more interesting: the `Applicative` instance.

```
src/HaskellBrainTeasers/WhatTheDuck/Instances.hs
instance Applicative List where
    pure a = List [a]
    List fs <*> List xs = List [f x | f <- fs, x <- xs]
```

The `Applicative` instance for lists includes an obvious definition of `pure`. The definition of `(<*>)` is slightly surprising at first but starts to make sense when you think about it more deeply. The first argument to `(<*>)` is a list of functions, and the second is a list of arguments to the functions. We combine the lists

by applying every function in the first list to every argument in the second list, in order. An example is worth a thousand lines of prose:

```
λ [ ("my "<>), ("peach "<>)] <*> ["pudding", "porridge"]
["my pudding", "peach pudding"
, "my porridge", "peach porridge"]
```

The Monad instance for lists follows the expected pattern. Instead of combining two existing lists, it combines nested lists:

```
src/HaskellBrainTeasers/WhatTheDuck/Instances.hs
```

```
instance Monad List where
```

```
  List xs >=> f = List $ concatMap (toList . f) xs
```

Like Functor, our Monad instance is trivial once we realize it mirrors a function we're already familiar with. In this case, ($>=>$) is simply `concatMap` with the order of the arguments reversed.

You might wonder why we'd bother using the Functor or Monad instances when we already have `map` and `concatMap`. You might prefer `fmap` over `map` for the flexibility to use arrays or vectors later. If your list is just an ordinary collection of values, you're more likely to use `concatMap` instead of ($>=>$).

It turns out that lists can be used for more than just a collection of arbitrary values. They can also represent a set of possible results in a nondeterministic computation. This view of lists reflects one of the traditional ways we use monads to represent computations. There are two common ways that you can model your programs as nondeterministic computations with lists: as a universe of potential solutions or as random variables.

The Many-Worlds Interpretation

One approach is to imagine your computation as a tree of potential values that you prune back to a set of valid results. This is the approach our puzzle uses to implement autocomplete. In this style, each time we use the binding arrow in a `do` block, we can imagine that we're working on the next set of intermediate values, or potential results, in parallel. Of course, not all values are valid, so we use the guard function to prune an entire subtree. We can see this in our puzzle:

```
step curChar curIdx = do
  (entryChar, entryIdx, entryOut) <- dict
  guard $ entryChar == curChar && entryIdx == curIdx
  nextChar <- entryOut
  result <- if nextChar == '$'
    then pure ""
    else step nextChar (curIdx + 1)
  pure $ curChar : result
```

In our step function, we can imagine that we're getting each character transition in parallel. We then prune out all subtrees that don't start with the right character at the right position. If we're in a valid state, we get the rest of the valid states recursively when we call `step` and then add our current character to the final result. Each path through the tree is like a separate potentially valid universe, each of which is represented in our list computation. As in our puzzle, this approach is most often useful for search problems where we want to represent a potentially large state space while avoiding unnecessary work.

The Copenhagen Interpretation

We can also represent nondeterminism more explicitly as random variables. In theory, we can have one element in the list for each potential result, duplicating values if we need to duplicate results. But in practice, we often model these programs as a list of values with a weight. For example, we can model a dice rolling program that returns the probability of all outcomes:

```
src/HaskellBrainTeasers/WhatTheDuck/DiceRoller.hs
type Distribution a = [(a, Double)]

uniformDistribution :: [a] -> Distribution a
uniformDistribution as = [(a, 1/count) | a <- as]
  where count = fromIntegral $ length as

normalize :: Ord a => Distribution a -> Distribution a
normalize rolls = do
  result <- groupBy (\(a,_) (b,_) -> a == b) $ sort rolls
  pure (fst $ head result, sum $ snd <$> result)

roll :: String -> Distribution Double
roll diceStr = normalize . foldl1' combineRolls $
  replicate count $ uniformDistribution [1..sides]
  where
    (read->count, read.tail->sides) = break (=='d') diceStr
    combineRolls roll1 roll2 = do
      (sideA, probA) <- roll1
      (sideB, probB) <- roll2
      pure (sideA + sideB, probA * probB)
```

In this approach, we can imagine each use of the binding arrow in our `do` block as sampling a random variable and combine these random variables using normal statistical reasoning.

Further Reading

Probability monad : <https://jelv.is/talks/probability-monad/slides.html#/sec-title-slide>

A Nice Recipe for Soup

src/HaskellBrainTeasers/Soup.hs

```
{-# LANGUAGE OverloadedStrings #-}  
{-# LANGUAGE GeneralizedNewtypeDeriving #-}  
{-# LANGUAGE DerivingVia #-}  
module HaskellBrainTeasers.Soup where  
import Data.String  
  
newtype Protein = Protein { getProtein :: String }  
    deriving (Show, IsString) via String  
  
newtype Veg = Veg { getVeg :: String }  
    deriving newtype (Show, IsString)  
  
newtype Broth = Broth { getBroth :: String }  
    deriving stock Show  
    deriving IsString via String  
  
data Soup = Soup Protein Veg Broth  
    deriving stock Show  
  
main :: IO ()  
main = print $ Soup "beans" "fennel" "tomato"
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

```
The program will output
Soup "beans" "fennel" (Broth {getBroth = "tomato"})
```

Discussion

This puzzle walks us through the different ways we can derive instances for our types—from soup to nuts. As an aperitif, we enjoy a taste of deriving an instance via a compatible type. Next, as an entrée, we enjoy deriving instances for a newtype using the instances of the type it wraps. As a main course, we get a double helping of deriving strategies with a stock instance of `Show` paired with a side of deriving via. The main dish amuses with a piquant surprise: In contrast to the other `Show` instances, our `Broth` comes with the type constructor and field name included. We finish with a stock `Soup` preparation. The result is a program that may not be a culinary masterpiece, but it gives us a *prix fixe* menu of options for how to get type class instances for free. Let's dig in to these different deriving strategies and learn how we can use them to write better more reusable programs.

Enriched Stock

In Haskell 98 and 2010, there are a few type classes that you're allowed to derive instances of automatically rather than by writing them yourself: `Eq`, `Ord`, `Read`, `Show`, `Enum`, and `Bounded`. These are often referred to as *stock derivable classes*. The syntax for deriving instances is one of the first things you learn about Haskell:

```
data Chowder = ClamChowder | CornChowder | FishChowder
  deriving (Eq, Ord, Read, Show, Enum, Bounded)
```

These stock derivable classes cover an unreasonably large number of use cases, but GHC has added language extensions over the years, to add both new stock derivable classes and new strategies for deriving instances. The new stock derivable classes have been largely unused in most code, with the exception of `DeriveGeneric`, which makes `Generic` a stock derivable class. New deriving strategies have gotten much more attention, with `anyclass` deriving seeing modest adoption for runtime generic programming, `GeneralizedNewtypeDeriving` being widely used for both simple domain types and *mtl*-style monad transformer stacks, and `DerivingVia` starting to gain adoption as the all-purpose multi-tool of deriving strategies. Let's take a look at each of these strategies and what they buy us.

Anytype Deriving

The `DeriveAnyClass` extension and corresponding `anyclass` deriving strategy make rather bold claims for what turns out to be a useful but rather modest feature. With the `anyclass` strategy, the compiler will attempt to generate an empty instance declaration for any type class you like. This works particularly well with runtime type classes that use runtime generic programming to provide default instances. The `aeson` package, for example, gives us `ToJSON` and `FromJSON` instances for free when we have an instance of `Generic`:

```
data SoupOrder =
  SoupOrder { soupType :: String, croutonCount :: Int }
  deriving stock Generic
  deriving anyclass (ToJSON, FromJSON)
```

Deriving Instances for Newtype Wrappers

The `GeneralizedNewtypeDeriving` extension lets us reuse an instance that is already defined for some type when we're creating a newtype wrapper. One of the most common ways that this extension is used today is with `mtl` to define custom monad stacks. For example, if we want to create a custom ordering action by stacking `StateT` and `ExceptT`, we can use the newtype deriving strategy to get direct access to the underlying `Functor`, `Applicative`, and `Monad` instances, along with any `mtl` classes:

```
newtype OrderM m dish =
  OrderM (StateT OrderState (ExceptT OrderError m) dish)
  deriving newtype
    ( Functor, Applicative, Monad
    , MonadState OrderState, MonadError OrderError)
```

One common mistake with `GeneralizedNewtypeDeriving` is that it can be used without specifying a deriving strategy, but any stock derivable instances will use the stock deriving strategy instead of the newtype strategy. This can lead to unexpected `Read` and `Show` instances. We can see this easily in `ghci`:

```
λ newtype MyInt = MyInt Int deriving (Num, Show)
λ 3 + 2 :: MyInt
MyInt 5
λ newtype MyInt = MyInt Int deriving newtype (Num, Show)
λ 3 + 2 :: MyInt
5
```

Since `Num` isn't a stock derivable class, we always fall back to `Int` for numeric operations. `Show` is a stock derivable, so we use the stock deriving strategy by default, but specifying the newtype strategy allows us to reuse the `Show` instance from `Int`.

Deriving via Representational Equality

The `DerivingVia` strategy takes the core idea behind `GeneralizedNewtypeDeriving` one step further. Instead of allowing us to reuse the instances from inside of a newtype wrapper, it lets us reuse instances between any types that share the same underlying representation.

Since `DerivingVia` works with any types that have the same representation, it can trivially replace any use of newtype deriving:

```
newtype MyInt = MyInt Int deriving (Num, Show) via Int
```

In this example, `MyInt` uses the underlying `Num` and `Show` instances from `Int` just as it did with the newtype strategy.

We can also use `DerivingVia` to replace `anyclass` deriving. One of the most common use cases for `anyclass` deriving is runtime generic programming, and the `GHC.Generics` module gives us the `Generically` type specifically for this use case with `DerivingVia`. Let's look at how we'd rewrite our earlier `SoupOrder` instances:

```
data SoupOrder =  
  SoupOrder { soupType :: String, croutonCount :: Int }  
  deriving stock Generic  
  deriving (ToJSON, FromJSON) via Generically SoupOrder
```

Here, instead of relying on an empty instance, we're able to take advantage of the fact that `ToJSON` and `FromJSON` both have instances for `Generically a`, which serve as blueprints for creating derived instances for any `Generic` type.

We can use this same technique to create blueprints for our own instances without relying on `Generic`. For example, let's say we want to create a reusable way of displaying redacted data in log messages. We start with a newtype, `Redacted`, that trivially wraps any other type. We give it the redacted `Show` instance and any other type can use it in place of the stock `Show`:

```
newtype Redacted a = Redacted a  
instance Show (Redacted a) where show _ = "redacted"  
data Password = Password { hash :: String, salt :: String }  
  deriving Show via Redacted Password
```

Further Reading

Deriving instances of extra classes in the GHC User's Guide

https://downloads.haskell.org/ghc/9.12.2/docs/users_guide/xts/deriving_extra.html

The Generic class

<https://hackage.haskell.org/package/base-4.21.0.0/docs/GHC-Generics.html#t:Generic>

Field Theory

```
src/HaskellBrainTeasers/RoundAndPointy.hs
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE DataKinds #-}
module HaskellBrainTeasers.RoundAndPointy where
import GHC.Records
import Text.Printf

data Rectangle = Rectangle Double Double deriving (Show)

instance HasField "area" Rectangle Double where
  getField (Rectangle w h) = w * h

instance HasField "pretty" Rectangle String where
  getField (Rectangle w h) = printf "a %f x %f rectangle" w h

newtype Circle = Circle Double deriving (Show)

instance HasField "area" Circle Double where
  getField (Circle r) = pi * r ** 2

instance HasField "pretty" Circle String where
  getField (Circle r) = printf "a circle of radius %f" r

showArea ::
  (HasField "area" r Double, HasField "pretty" r String) =>
  r -> String
showArea r = printf "The area of %s is %f" r.pretty r.area

main :: IO ()
main = do
  putStrLn . showArea $ Rectangle 3 5
  putStrLn . showArea $ Circle 2.5
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

```
The program will output
The area of a 3.0 x 5.0 rectangle is 15.0
The area of a circle of radius 2.5 is 19.634954084936208
```

Discussion

Oops, this function seems to have gotten its objects mixed into our functions. Not to worry, though—we haven’t compromised our pure functional program, we’ve just used the `OverloadedRecordDot` extension to enable *record dot syntax*. Record dot syntax solves a long-standing annoyance for many Haskell users who didn’t feel that the dot (`.`) character was used enough, being historically relegated to obscure things like function composition, universal quantification in type annotations, hierarchical module names, and import qualification. As a bonus, record dot syntax also lets us access record fields. In our puzzle, we’re using record dot syntax to access virtual record fields.

The `OverloadedRecordDot` extension has only been around since GHC 9.2, but record dot syntax is really just a lightweight layer of syntactic sugar over features that GHC has had for quite some time—most importantly, the `OverloadedRecordFields` that landed in GHC 8.2 and significantly extended our ability to work with polymorphic record fields.

Let’s take a deeper look at records and some of the tools GHC gives us for working with them, and then we can see what record dot syntax gives us and what happens if we follow record dot syntax to its natural conclusion.

Dualing Banjos

Normally, we think of data types in Haskell as something we construct with values. We might, for example, construct a Banjo by passing in a style, a number of strings, and how it’s tuned:

```
data Banjo = Banjo
    {style :: String, strings :: Int, tuning :: String}

bluegrassBanjo :: Banjo
bluegrassBanjo = Banjo "Bluegrass" 5 "Open G"

formatBanjo :: Banjo -> String
formatBanjo (Banjo style strings tuning) =
    printf "A %d-string (%s) %s banjo" style tuning strings
```

As ordinary Haskellers, we might call Banjo a data type, but if we listen quietly, we can hear the voices of the top hatted and bemonocled category theorists whispering that “Banjo is an object in the category of data types.” Unfortunately,

the distant whispers of the category theorists are often drowned out by roving gangs of Agile software architects who, upon hearing the word *object*, have the urgent need to explain that accessing fields directly will lead to failing tests and utter chaos and will demand that we wrap those fields with getter methods.

Field accessors are already functions that retrieve values from a record, but if we're feeling a bit smarmy, we might decide to see how far we can take the idea. Pretty far, it turns out, because any data we can represent by passing values into a constructor can also be represented by a set of functions that extract values from some opaque value. If you listen closely, you can hear our category theorists whispering the word *codata* into the winds, but looking at a concrete example turns out to be more helpful:

```
class ClassyBanjo a where
  getStyle :: a -> String; getStrings :: a -> Int; getTuning :: a -> String
data BluegrassBanjo = BluegrassBanjo
instance ClassyBanjo BluegrassBanjo where
  getStyle _ = "Bluegrass"; getStrings _ = 5; getTuning _ = "Open G"
```

In this example, `ClassyBanjo` provides functions that we can use to extract information about a banjo from some value. This pattern, where a datatype is defined not by how we construct it but instead by how we extract values from it, is called *co-data*. Thanks to field selectors and pattern matching, all data in Haskell is also co-data, but this type class-based approach leans even further into co-data and gives us something that looks more like a traditional object-oriented design.

Unfortunately, this approach to co-data still has some limitations. Notably, since we've bundled all the fields together into a single type class, we're still restricted to calling `getStrings` or `getTuning` on banjo-like types. Using type classes technically gives us polymorphism, but in practice the restrictions we have are analogous to the restriction on duplicate field selector names in records.

One way to address this problem is to define a separate type class for each record field. The `HasField` type class from `GHC.Record` gives us a convenient way to create an instance for a particular record field:

```
instance HasField "fieldName" SomeDataType FieldType where
  getField SomeDataType = FieldType
```

This instance tells us that the data type `SomeDataType` has a field named `fieldName` of type `FieldType`. In our puzzle, we wrote `HasField` instances to add `area` and `pretty` fields to both of our shape types. Hand-written instances like this that don't reflect actual fields in underlying data types are called *virtual fields* and,

much like getter methods in object-oriented programs, can be useful during refactoring for backwards compatibility and API interoperability.

Most of the time when we're using `HasField`, we're not dealing with virtual fields. Instead, we want a simple polymorphic way to access fields with the same name when they're part of different data types, even when the fields themselves have different types. Thankfully, in these cases, GHC saves us from needing to write instances by hand. Since GHC 8.2, exported field selectors can be used to solve a `HasField` constraint, so you can use `getField` as a more polymorphic alternative to normal field selectors without needing to write instances yourself.

Dot Products

Understanding co-data and the `HasField` class gets us most of the way toward understanding record dot syntax. The `OverloadedRecordDot` extension simply adds a small amount of syntactic sugar to `HasField` by allowing us to type `a.b` instead of `hasField b a`. This works whether or not we've imported the `GHC.Records` module.

This syntactic sugar comes with two significant costs. First, with `OverloadedRecordDot` enabled, we need to use whitespace to differentiate between function composition and field access. Since the extension changes the grammar slightly, adding it to an existing program may cause your program to stop compiling. Or, in very rare circumstances, it may change the semantic meaning of your program. Second, using `OverloadedRecordDot` and co-data can make Haskell feel much more like a traditional object-oriented language. This can have ergonomic benefits but can also encourage unidiomatic code. In fact, when taken to the extreme, `OverloadedRecordDot` can help us write code that hardly looks or behaves like Haskell at all, as shown in the code on the facing page.

Further Reading

Overloaded record fields

<https://gitlab.haskell.org/ghc/ghc/-/wikis/records/overloaded-record-fields/>

src/HaskellBrainTeasers/RoundAndPointy/Append.hs

```

newtype Stack a = Stack (IORef [a])
stack :: Stack a; stack = undefined
instance HasField "new" (Stack a) (() -> IO (Stack a)) where
  getField _ _ = Stack <$> newIORef []
instance HasField "push" (Stack a) (a -> IO ()) where
  getField (Stack ref) nextVal = modifyIORef ref (nextVal :)
instance HasField "pop" (Stack a) (() -> IO a) where
  getField (Stack ref) () = do
    (x:xs) <- readIORef ref; writeIORef ref xs; pure x
instance HasField "forEach" [String] ((String -> IO ()) -> IO ())
  where getField inputs f = for_ inputs $ \input -> f input
rpn :: [String] -> IO Int
rpn inputs = do {
  s <- stack.new();
  inputs.forEach (\input -> case input of
    "+" -> do {a <- s.pop(); b <- s.pop(); s.push(b + a);}
    "*" -> do {a <- s.pop(); b <- s.pop(); s.push(b * a);}
    "-" -> do {a <- s.pop(); b <- s.pop(); s.push(b - a);}
    "/" -> do {a <- s.pop(); b <- s.pop(); s.push(b `div` a);}
    number -> s.push(read number)
  );
  s.pop();
}

```

Part IV

The Type System

The world has gone typesy-turvy in these mind-twisting puzzles that deal with Haskell's type system. These puzzles will challenge you to rank up your types, let fly with linear arrows, and even stealthily skirt the dreaded monomorphism restriction. As you work through these puzzles, remember to check the type and kinds of your expressions in ghci with :type and :kind!, and don't forget that visible type applications can help you better understand types interactively. If you find that a puzzle is too difficult, skip it and try again later or read the detailed explanation and try to come up with your own example that demonstrates the behavior.

Round-Trip Tickets

`src/HaskellBrainTeasers/RoundTripTicket.hs`

```
module HaskellBrainTeasers.RoundTripTicket where

roundTrip ::
  (Show a, Show b) => String -> a -> b -> String
roundTrip maxLenStr a b =
  take maxLen $ show (hither $ thither a, hither $ thither b)
  where
    maxLen = hither maxLenStr
    hither = read
    -- refactored from @thither x = show x@ to avoid hlint
    -- warnings to eta reduce
    thither = show

main :: IO ()
main = putStrLn $ roundTrip "5" 10 10
```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

```

The program will fail to compile with an error:
/tmp/RoundTripTicketXXX122964-1.hs:9:60: error: [GHC-25897]
" Couldn't match expected type 'a' with actual type 'b'
  'b' is a rigid type variable bound by
    the type signature for:
      roundTrip :: forall a b.
        (Show a, Show b) =>
          String -> a -> b -> String
    at /tmp/RoundTripTicketXXX122964-1.hs:(6,1)-(7,48)
'a' is a rigid type variable bound by
  the type signature for:
    roundTrip :: forall a b.
      (Show a, Show b) =>
        String -> a -> b -> String
    at /tmp/RoundTripTicketXXX122964-1.hs:(6,1)-(7,48)
" In the first argument of 'thither', namely 'b'
   In the second argument of '($)', namely 'thither b'
   In the expression: hither $ thither b
" Relevant bindings include
   thither :: a -> String
     (bound at /tmp/RoundTripTicketXXX122964-1.hs:15:5)
   b :: b (bound at /tmp/RoundTripTicketXXX122964-1.hs:8:23)
   a :: a (bound at /tmp/RoundTripTicketXXX122964-1.hs:8:21)
   roundTrip :: String -> a -> b -> String
     (bound at /tmp/RoundTripTicketXXX122964-1.hs:8:1)
9 |   take maxLen $ show (hither $ thither a, hither $ thither b)
   |                                                              ^

```

Discussion

In this puzzle, a well-intentioned refactor has caused havoc by summoning the dread monomorphism restriction. The monomorphism restriction, and its close friend the ambiguous type error, is the cause of many completely safe refactors that are immediately followed by CI failures. Thankfully, by carefully studying the nuances of the monomorphism restriction and how type class resolution works, you can confidently continue to make this exact mistake for the rest of your career.

Let's start by taking a look at what the monomorphism restriction is and when it applies to your code. Afterwards, we'll look at a few common ways that you can accidentally trip over it in your programs and what to look for.

What is the Monomorphism Restriction?

The monomorphism restriction prevents GHC from inferring polymorphic types in some cases. Although the specifics of the grammar and when the restriction applies are outlined in the Haskell 2010 Report, the simplest way to understand it is that it applies when you bind variables in `let` expressions and `where` clauses. One of the most surprising things about the monomorphism restriction is that it only applies to bindings without arguments to the left of the equal sign. It's much easier to understand in code, so let's look at a few examples:

```
-- monomorphism restriction applies because value is an ordinary binding
let value = somethingCool "George" "Spaceships" in ...

-- monomorphism restriction doesn't apply, cases aren't simple bindings
case somethingCool "George" "Spaceships" of { value -> ... }

-- monomorphism restriction applies, increment is a simple function binding
let increment = (+1)
let increment = \n -> 1 + n

-- monomorphism restriction doesn't apply, there is a type annotation
let
  increment :: Num a => a -> a
  increment = (+1)

-- monomorphism restriction doesn't apply, arguments appear left of =
let increment n = 1 + n
let increment _unused = \n -> 1 + n
```

Notably, the monomorphism restriction means that eta reduction or eta expansion (fancy-people speak for adding or removing arguments) can change whether your program compiles:

```
-- monomorphism restriction applies
let display = show
let parse = read

-- monomorphism restriction doesn't apply
let display val = show val
let parse input = read input
```

Somewhat less obviously, the monomorphism restriction is also disabled when you turn on the `NoMonomorphismRestriction` language extension.

Why Is It a Problem?

The most subtle problem with the monomorphism restriction is that innocuous and seemingly stylistic refactors can cause your program to suddenly fail to compile. If you do somehow notice that your program has failed to compile, you may also notice that the errors have an unfortunate tendency to say

nothing about the monomorphism restriction. Instead, the monomorphism restriction tends to cause type unification and ambiguous type errors.

Type unification errors most often occur when we eta reduce and suddenly find out that our newly pointfree code has chosen violence:

```
showPair :: (Show a, Show b) => a -> b -> (String, String)
showPair a b = (display a, display b)
  where display = ("displays as: " <>) . show
```

Since `display` has no arguments to the left of the equal sign, the monomorphism restriction kicks in and says that we have to pick a single concrete type. We call it with `a` first, so the compiler infers the type `display :: a -> String`. When we try to call `display b`, the compiler helpfully lets us know that it can't prove that `a` is equal to `b`. If we rewrite this in a less pointless style with an argument on the left-hand side, then the monomorphism restriction no longer applies and the code works as expected:

```
where display x = "displays as: " <> show x
```

Programmers, of course, hate nothing more than consistency. As you might expect, eta expansion can also confound the monomorphism restriction. Instead of type equality errors, eta expansion tends to result in ambiguous type errors:

```
showMore :: (Monoid a, Show a) => a -> String
showMore a = display a <> display mempty
  where display x = show x
```

In this example, having faithfully learned the lesson that we should always eta expand, we've created a pointful version of `display` with an inferred type of `display :: Show a => a -> String`. Since the function is polymorphic, each time we call it the compiler will try to pick out the appropriate type. Our first call is at type `a`, which has a `Show` instance, and so everything goes as expected. Our second call passes in `mempty`, but that could be any type with a `Monoid` instance. Since `display` is polymorphic, our earlier call does nothing to narrow down which type we're trying to use, and the compiler eventually gives up and asks for help. With a quick and apparently not so pointless pointfree refactor, the monomorphism restriction kicks in and saves our program:

```
where display = show
```

Real-World Tips and Tricks

The examples of eta expansion and reduction we've seen aren't entirely contrived. These sorts of problems happen all the time during real-world refactors. One of the most common scenarios is when you factor something out of a let

or where binding to the top level and add an argument to pass in a value that had previously been in scope. That argument disables the monomorphism restriction, and suddenly you find yourself with bonus type errors.

There are, thankfully, better ways of addressing this problem than stochastically permuting between pointfree and pointful styles. Visible type applications, for example, can neatly solve the problem of ambiguity:

```
showMore :: forall a. (Monoid a, Show a) => a -> String
showMore a = display a <> display (mempty @a)
  where display x = show x
```

Further Reading

The monomorphism restriction in the Haskell 2010 Report : <https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-930004.5.5>

The monomorphism restriction on the Haskell wiki : https://wiki.haskell.org/index.php?title=Monomorphism_restriction

The MonoLocalBinds extension : https://ghc.gitlab.haskell.org/ghc/doc/users_guide/extends/let_generalisation.html

The NoMonomorphismRestriction extension : https://ghc.gitlab.haskell.org/ghc/doc/users_guide/extends/monomorphism.html#extension-MonomorphismRestriction

A Grand Un-Unified Theory

```
src/HaskellBrainTeasers/Salad.hs
{-# LANGUAGE DerivingStrategies #-}
module HaskellBrainTeasers.Salad where
import Data.List
import Data.Functor.Identity

newtype Secret s a = Secret (Identity a)
    deriving newtype (Functor, Applicative, Monad)

getSecret :: (forall s. Secret s a) -> a
getSecret (Secret (Identity a)) = a

data Ingredient a where
    Oil :: Ingredient String
    Acid :: Ingredient String
    Herbs :: Ingredient [String]

newtype Recipe s = Recipe (forall a. Ingredient a -> a)

withRecipe ::
    Recipe s -> ((forall a. Ingredient a -> Secret s a) -> r) -> r
withRecipe (Recipe get) cook = cook (pure . get)

vinaigrette :: Secret s (Recipe s)
vinaigrette = pure $ Recipe $ \i -> case i of
    Oil -> "Olive Oil"
    Acid -> "Red Wine Vinegar"
    Herbs -> ["Basil", "Cilantro"]

advancedPrep :: Secret s (Recipe s) -> String
advancedPrep secretRecipe = getSecret $ do
    recipe <- secretRecipe
    withRecipe recipe $ \get -> do
        acid <- get Acid; oil <- get Oil; herbs <- get Herbs
        pure $ "combine: " <> intercalate ", " (acid:oil:herbs)

main :: IO ()
main = putStrLn $ advancedPrep vinaigrette
```

Guess the Output



Before moving on to the next page, try to guess the output. Do you think this program will complete at all? Will it complete quickly?

```

The program will fail to compile with an error:
/tmp/SaladXXX122964-2.hs:34:13: error: [GHC-25897]
    " Couldn't match type 's1' with 's'
      Expected: Secret s1 (Recipe s)
      Actual:   Secret s (Recipe s)
    's1' is a rigid type variable bound by
      a type expected by the context:
        forall (s1 :: k). Secret s1 String
    at /tmp/SaladXXX122964-2.hs:(33,41)-(37,59)

```

Discussion

In this puzzle, we're trying to create a secret salad dressing recipe, but unfortunately our types refuse to emulsify... err, *unify*. Our recipe doesn't call for Gorgonzola, but we still have something rank: higher-ranked types. We're making use of higher-ranked types for two different purposes. First, we're combining them with generalized algebraic data type (GADT) refinement to build functions that let us work with purely symbolic ingredients. Second, we're using a higher-ranked phantom type to ensure that we can't accidentally leak data outside of a `Secret` computation. Let's take a look at what higher-ranked types are, then look in more detail at how we're using them in our puzzle.

First-er Class Polymorphism

The easiest way to understand higher-ranked polymorphism is to think of it as a way of describing how many layers of polymorphism a type contains. Like cheese, the higher the rank, the more intense and aromatic the polymorphism. Let's take a look at a few functions of increasingly ranked polymorphism and compare them to get a better idea of what this means in practice.

```

roquefort :: forall x. x -> [x]
roquefort x = [x,x]

limburger :: (forall x. x -> [x]) -> Int
limburger makeList =
  length (makeList "Camembert") + length (makeList 10)

epoisses :: ((forall x. x -> [x]) -> Int) -> Int
epoisses listMaker = listMaker roquefort + listMaker stilton
  where
    stilton :: forall y. y -> [y]
    stilton y = [y,y,y]

```

Our first function, `roquefort`, is an ordinary rank 1 polymorphic function. It has a single “layer” of polymorphism: The type variable `x` represents some concrete but unknown type that will be selected by the caller. Since we don’t know which type will be selected, we’re limited in how we can define the function (this property is called *parametricity*). In fact, the only valid definitions of `roquefort` are those that return lists containing different numbers of the value `x`.

Moving one step up the rankness ladder, we have `limburger` with a rank 2 type. The argument we pass `limburger` must itself be a rank 1 polymorphic function and `limburger` gets to pick the type of values it wants to pass into that function each time it’s called. In our example, `limburger` calls `makeList` first with a `String` and later with an `Integer`. This demonstrates that `makeList` is polymorphic, but the polymorphism also makes the values we’re passing in superfluous. Since `makeList` can’t possibly inspect or do anything else with the value we pass it (thanks to parametricity), we could just pass undefined to `makeList` and get the same result.

At the top of our rankness ladder is `epoisses`, a function with a rank 3 type. This function accepts a rank 2 polymorphic function as an argument, continuing the same pattern we’ve seen in our other functions. We, the caller, picked some “rank 0” (monomorphic) type and used it when we called rank 1 `roquefort`. We picked a rank 1 function and passed it to rank 2 `limburger`, which in turn picked some monomorphic types to call that function with. Now, with `epoisses`, we pick some rank 2 function that `epoisses` calls with some rank 1 types.

Although our examples only go up to rank 3, GHC supports arbitrarily ranked types with the `RankNTypes` language extension (enabled by default in GHC2021 and GHC2024). The same pattern you’ve seen up to rank 3 continues as you increase the rank of your types. In practice, rank 1 types are ubiquitous in Haskell. Rank 2 and 3 types are somewhat advanced, but as we’ll see, they have several concrete use cases. Types of rank 4 and above are relatively rare in practice.

Omnivorous GADT Consumers

In our puzzle, `withRecipe` shows us one of the most interesting uses of higher-ranked types: working with GADTs. Standard algebraic data types, even when they have type parameters, tend to work naturally with rank 1 polymorphism because programs tend to work only with one type of value at a time. With GADTs, each constructor may generate a different type of value. This can be a problem when we want to write functions where different branches could generate different GADTs. For example, imagine we’re writing a small DSL that lets us represent either numbers or strings:

```
data Expr a where
  ENum :: Int -> Expr Int; EStr :: String -> Expr String
```

An astute reader may realize that `Expr` could be represented more simply as `Either Int String`. Such astute readers should check under their sofas and between the cushions, and resume reading once they have located their pedagogical suspension of disbelief.

If we wanted to parse an ordinary string into an `Expr`, with numeric strings becoming `ENum` values and other strings becoming `EStr` values, we might start with something straightforward like this:

```
parseExpr :: String -> Expr n
parseExpr (readMaybe -> Just n) = ENum n
parseExpr s = EStr s
```

Like many things in the world, this function is simple, easy to read, and wrong. This rank 1 polymorphic function allows the caller to select the type of `Expr` that should be returned, but, depending on the string, we might not return the desired type. Variations of this problem come up frequently when using GADTs, and the solution turns out to be that instead of trying to return a GADT directly, we need to ask the caller to pass us a polymorphic continuation function that can work with all possible GADT values:

```
withExpr :: String -> (forall n. Expr n -> r) -> r
withExpr (readMaybe -> Just n) go = go (ENum n)
withExpr s go = go (EStr s)
```

With this approach, we can construct a GADT of the appropriate type as necessary, without the caller being able to control which GADT value should be constructed, and we can ensure that we have a total function that will always let us get back a value of some specific caller-determined return type. In our puzzle, we use this same approach to work with GADTs, but we take it one step further by turning the polymorphic function that fetches a value for all ingredients into a proper data type: *Recipe*.

The ST Trick

Perhaps the most common use of higher-ranked types in most real-world Haskell code is what most people call “the ST trick” (or, if you’re still wearing your top hat and monocle, resource management by way of *Skolemization*).

The idea comes from the ST monad, which uses higher-ranked types to manage resources, allowing you to write code that internally uses mutable data but externally looks and behaves like a pure function. Our puzzle uses the same technique to lock up our *Secret* values, and our misuse of secret values is why the puzzle fails to compile.

To make use of the trick, we need to do two things: First, we need to create a type that tracks its resource with a phantom type—`s`, in our example. Our puzzle lives in a single file, but in practice this technique uses smart constructors to ensure that the only way to get a value out of a `Secret` or `ST` is through a rank 2 function—in our case, `getSecret`:

```
newtype Secret s a = Secret (Identity a)
  deriving newtype (Functor, Applicative, Monad)
getSecret :: (forall s. Secret s a) -> a
getSecret (Secret (Identity a)) = a
```

Since `getSecret` must work for all secrets, we can never assume any one particular secret. When we were working with GADTs, we handled this with continuation passing and pattern matching on GADT constructors. Since `s` is a phantom type, there's nothing we can do to refine our understanding of `s`, and every single `Secret s` must be treated as unique (in other words, no two `s` values can unify). The compiler manages this by generating a unique type for `s` with Skolemization.

The compiler error arises because `advancedPrep` is a rank 1 polymorphic type, meaning that the caller gets to pick a particular type for `s`. When we try to use `secretRecipe` inside our `do` block, the compiler reasons that the caller-provided type of `s` is different from the brand-new type generated by `getSecret` and fails. We are essentially trying to combine values with types `Secret callerSecret` and `Secret brandNewSecret`, and they don't line up.

Once we understand the reason for the failure, fixing it is relatively easy. We can avoid calling `getSecret` in `advancedPrep` and instead push that to the caller so that `getSecret` gets to pick the type of `s`:

```
main = putStrLn $ getSecret (advancedPrep vinaigrette)
```

Alternatively, we can give `advancedPrep` a rank 2 type so that the caller doesn't get to pick the type of `s`:

```
advancedPrep :: (forall s. Secret s (Recipe s)) -> String
```

Further Reading

Arbitrary rank polymorphism in the GHC User's Guide

https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/rank_polymorphism.html

The ST monad

<https://hackage.haskell.org/package/base-4.21.0.0/docs/Control-Monad-ST.html>

Lazy Functional State Threads

<https://dl.acm.org/doi/10.1145/178243.178246>

Non-Functional Dependencies

```

{-# LANGUAGE DataKinds, GADTs #-}
{-# LANGUAGE RequiredTypeArguments, UndecidableInstances #-}
module HaskellBrainTeasers.NonFunctionalDependencies where
import GHC.TypeLits
import Text.Printf
import Data.List (intercalate)

type MissingKey k v = TypeError
  (Text "No key " :<: ShowType k :<: Text " of type " :<: ShowType v)

data KVStore elems where
  Empty :: KVStore '[]
  Cons :: KnownSymbol k => v -> KVStore elems -> KVStore ('(k,v):elems)

class HasKey kvStore key valType where
  (!) :: forall k -> k ~ key => kvStore -> valType
instance MissingKey key val => HasKey (KVStore '[]) key val where
  _ ! _ = error "missing key"
instance {-# OVERLAPPABLE #-}
  HasKey (KVStore xs) key val =>
    HasKey (KVStore ('(badKey,badVal):xs)) key val where
  k ! Cons _ rest = k ! rest
instance HasKey (KVStore ('(key,val):rest)) key val where
  _ ! Cons v _rest = v

main :: IO ()
main = putStrLn $
  printf "I'd like a %s crust with %s, and %s cheese"
    ("crust" ! pizza)
    (intercalate ", " $ "toppings" ! pizza)
    ("cheese" ! pizza)
  where
    pizza =
      Cons @"crust" "thin" $ Cons @"cheese" "provel" $ Cons @"slices" 25 $
      Cons @"toppings" ["pepperoni", "mushroom", "onion"] Empty

```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

```

The program will fail to compile with these errors:
/tmp/NonFunctionalDependenciesXXX122964-3.hs:32:3: error: [GHC-39999]
  Ambiguous type variable 't0' arising from a use of 'printf'
/tmp/NonFunctionalDependenciesXXX122964-3.hs:33:14: error: [GHC-43085]
  Overlapping instances for HasKey
/tmp/NonFunctionalDependenciesXXX122964-3.hs:35:15: error: [GHC-43085]
  Overlapping instances for HasKey
/tmp/NonFunctionalDependenciesXXX122964-3.hs:38:71: error: [GHC-39999]
  Ambiguous type variable 'v0' arising from the literal '25'

```

Discussion

In this puzzle, we're trying to order dinner and, like a square-cut pizza topped with Provel, the compiler is serving us nothing but disappointment. We start by creating a key-value store, `KVStore`, that acts like a dependent map, where each key determines the type of the associated value. Knowing that it's best to reserve advanced techniques such as this for situations that demand the utmost correctness, we're using our dependent map to store ingredients for a pizza (or, more precisely, ingredients for the best pizza).

We construct a `KVStore` by calling `Cons` directly, passing in type-level keys and term-level values. Each time we insert a new value, it's added to the type-level list of key-value pairs. We can access values using the `(!)` operator we've defined in the `HasKey` type class. `HasKey` is a recursive type class that steps through each element in our key-value store. If the key we're looking for is at the head of the list, we return the associated value; otherwise, we search the tail of the list. If we're ever searching for a key in an empty list, it means we were looking for something that didn't exist, and so we generate a `TypeError`.

Our program is short, elegant, and fails to compile. The core problem is that, without more information, the compiler can't figure out what type of value we're getting from `KVStore`. Let's take another look at our instance declaration:

```
instance HasKey (KVStore ('(key,val):rest)) key val where
```

We can see the structure that says each `val` depends on the key we look up and the `KVStore` we're looking it up in, but the compiler looks at our instance with confusion and says, "Okay, tell me which `val` you want, and I'll see if I can find an instance." Unless we provide a specific type for `val`, the compiler can't figure out which instance to call, resulting in the *overlapping instances* error. And it doesn't have a concrete type to use when solving the constraints for our call to `printf`, resulting in the *ambiguous type variable* errors.

We can solve the problem by making the value type depend on the types of the KVStore and the key we're looking up. Passing type theorists may stop to note that the term for this is *injectivity*. If they do, please give them a slice of pizza and send them back to their university or research lab.

Associated type families are generally the preferred way to add injective types to type classes. They let us create a new data type that needs to be defined for every instance of a class, and they provide a type-level function from the instance type to the associated type. As an example, let's imagine an Indexable type class for types that we can index into. Naively, we may try to implement this with standard multiparameter type classes:

```
class Indexable t a where index :: Int -> t -> a
instance Indexable String Char where index idx str = str !! idx
firstTwo :: String -> String
firstTwo str = printf "%C%C" (index 0 str) (index 1 str)
```

This example neatly reproduces the same ambiguous type error problem of our original puzzle. We know that Indexable String Char is the only instance we've defined, but, from the compiler's perspective, there might be other instances lurking around. In theory, we could have Indexable String () or Indexable String Int lurking around one module over. Since printf is also polymorphic, it doesn't demand a specific type that would let the compiler know for certain which instance to choose, and so the overall expression is ambiguous.

Using associated type families, we avoid this problem by removing the second parameter from our type class. Instead, we create a new Element type inside of the type class definition. This requires that each instance—String, in our example—define its associated element type.

```
class Indexable t where
  type Element t :: Type; index :: Int -> t -> Element t
instance Indexable String where
  type Element String = Char; index idx str = str !! idx
```

This version of our program compiles and runs as expected. Unlike the earlier version, we don't have any ambiguity. The compiler knows that we must be using the String instance of Indexable, and that means index must be returning a Char.

Associated type families are, generally, a good way to encode injective types when working with type classes, but unfortunately they don't always work out. Let's try to use an associated type family with our puzzle:

```
class HasKey kvStore key where
  type Val kvStore key :: Type
  getKey :: forall k -> k ~ key => kvStore -> Val kvStore k
```

```
instance {-# OVERLAPPABLE #-} HasKey (KVStore xs) key =>
  HasKey (KVStore ('(badKey,badVal):xs)) key where
  type Val (KVStore ('(badKey,badVal):xs)) key = Val (KVStore xs) key
  getKey key (Cons _ rest) = getKey key rest
instance HasKey (KVStore ('(key,val):rest)) key where
  type Val (KVStore ('(key,val):rest)) key = val
  getKey key (Cons val _rest) = val
```

This version tries to replace the `val` parameter in our type class with an associated type family, but it fails with an error about “conflicting family instance declarations.” The problem is that we’ve designed our type class and instances to overlap, letting us recursively search through our key-value store until we get to an instance with the right key at the head of the `KVStore` list. Although we may be able to lean more heavily into type families to restructure our code to avoid this problem, that requires a significant refactor and could greatly increase both the time and memory requirements to compile our program.

Thankfully, the `FunctionalDependencies` extension gives us another way to encode injectivity. We continue to track `valType` as a type class parameter, but now we can explicitly tell the compiler that it depends on `kvStore` and `Key`:

```
class HasKey kvStore key valType | kvStore key -> valType where
  (!) :: forall k -> k ~ key => kvStore -> valType
instance MissingKey key val =>
  HasKey (KVStore '[]) key (Proxy (KVStore '[], key)) where
  _ ! _ = error "missing key"
instance {-# OVERLAPPABLE #-}
  HasKey (KVStore xs) key val =>
  HasKey (KVStore ('(badKey,badVal):xs)) key val where
  k ! Cons _ rest = k ! rest
instance HasKey (KVStore ('(key,val):rest)) key val where
  _ ! Cons v _rest = v
```

In this example, we use the `| kvStore key -> valType` syntax to tell the compiler that the combination of `kvStore` and `key` must always determine the type for `valType`. Thanks to the injectivity constraint, we no longer have any ambiguity. And unlike our attempt that used associated type families, we can still take advantage of overlapping instances. Unfortunately, as you may have noticed, the injectivity requirement with functional dependencies can come at a cost—in this case, the error-handling base case now requires that we artificially choose a `valType` that depends on `KVStore '[]` and `key`. A type-level list or tuple can be a good choice to work around the constraint in these situations. The specific type you pick doesn’t matter so long as it mentions all of the types that determine the injective type.

Further Reading

Type families in the GHC User's Guide

https://downloads.haskell.org/ghc/latest/docs/users_guide/xts/type_families.html#type-families

Functional dependencies in the GHC User's Guide

https://downloads.haskell.org/ghc/latest/docs/users_guide/xts/functional_dependencies.html#functional-dependencies

Instance declarations and resolution in the GHC User's Guide

https://downloads.haskell.org/ghc/latest/docs/users_guide/xts/instances.html

Time Flies Like an Arrow

```

{-# LANGUAGE LinearTypes #-}
module HaskellBrainTeasers.TimeFliesLikeAnArrow where
import Prelude.Linear qualified as L

newtype Banana where Banana :: String %1 -> Banana
newtype FruitFly where FruitFly :: String %1 -> FruitFly

aBanana :: Banana
aBanana = Banana "Cavendish"

mkFruitFly :: Banana %1 -> FruitFly
mkFruitFly (Banana b) = FruitFly b

prettyprintFruitFly :: FruitFly %1 -> String
prettyprintFruitFly (FruitFly s) = "Fruit Flies like a " L.<> s

main :: IO ()
main = do
    putStrLn "Time Flies like a Linear Arrow"
    putStrLn (prettyprintFruitFly $ mkFruitFly aBanana)

```

Guess the Output



Before moving on to the next page, try to guess the output. Will it compile? Will it finish running?

```

The program will fail to compile with an error:
/tmp/TimeFliesLikeAnArrowXXX122964-4.hs:22:13: error: [GHC-83865]
    " Couldn't match type 'One' with 'Many'
      Expected: FruitFly -> String
      Actual:   FruitFly %1 -> String
    " In the first argument of '($)', namely 'prettyprintFruitFly'
      In the first argument of 'putStrLn', namely
        '(prettyprintFruitFly $ mkFruitFly aBanana)'
      In a stmt of a 'do' block:
        putStrLn (prettyprintFruitFly $ mkFruitFly aBanana)
22 |     putStrLn (prettyprintFruitFly $ mkFruitFly aBanana)
    |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Discussion

Multiplicity is a 1996 comedy starring Michael Keaton and directed by Harold Ramis. In the film, a man begins cloning himself to deal with the unsustainable number of obligations in his life. The multiplicity errors in our puzzle have nothing to do with the film—quite the opposite in fact. While the film features hijinks caused by multiple copies of a single man, our puzzle fails because we’ve used *linear types* without carefully proving to the compiler that we’ll use the value exactly once.

The problem is that we’re mixing a linear function, `prettyprintFruitFly`, with a function that doesn’t know anything about linear types—in this case, the function application operator `($)`. Even though we can look at the code and see that it should work, we’re trying to mix incompatible universes.

There are two easy fixes for our puzzle. We can either use parentheses instead of `($)` or use the linear-aware version of `($)` from `Prelude.Linear`:

```

-- L.$ works with both linear and unrestricted functions
putStrLn L.$ prettyprintFruitFly L.$ mkFruitFly aBanana

-- ($) only works with unrestricted functions
putStrLn $ prettyprintFruitFly (mkFruitFly aBanana)

```

This solves the immediate problem, but without understanding why we ran into this problem, we’ll run into it repeatedly and struggle to make progress with linear types. In the rest of this discussion, let’s dig into how the `LinearTypes` extension works, what it means for how we write code, and why all the trouble is worth it.

Sunshine, Lollipops, and Arrows Everywhere

Haskell’s type system is based (roughly) on a type theory called System F, which has a correspondence to formal logic and tells us how we can reason about what types are and what they mean. There are certain rules that come along with the structural logic that Haskell’s type system is built on. In logic land, for example, we might say that we’re allowed to add new assumptions to an already valid conclusion (this is called *weakening*) or that if we are making an assumption twice, we can instead just make the assumption once (this is called *contraction*). Several glasses of type-theory-flavored punch later and you might realize that we can demonstrate these rules with very simple Haskell functions:

```
weaken :: (a -> c) -> a -> b -> c; weaken f a _ = f a
contract :: (a -> a -> b) -> a -> b; contract f a = f a a
```

In Haskell land, weakening says that we can ignore terms if we’d like, and so we can always add extra ignored terms to any expression and still end up with something that type checks. Contraction, on the other hand, says that if we use a term twice, we can always write an expression that uses the term once instead—we just need to speak the incantation “referential transparency” while we duplicate the value.

These rules are so intuitive (*intuitionistic* for any roving logicians) that we might assume they’re mandatory for a programming language to work. But it turns out that if we prohibit both weakening, which allows us to have variables we don’t use, and contraction, which lets us use variables more than once, we end up with linear variables that we must use exactly once. Variables that we can weaken or contract are called *unrestricted*.

In GHC, we can track whether a particular variable is linear or unrestricted by its multiplicity. Linear variables have a multiplicity of One (often written %1), and unrestricted variables have a multiplicity of Many.

If we write ordinary Haskell code without thinking about multiplicity at all, we’ll typically end up working with unrestricted variables and never have to think about linearity at all. Enabling the LinearTypes extension allows us to explicitly write multiplicity annotations for functions as well as record fields and let and where bindings. Let’s look at a couple of examples:

```
-- GHC lets us explicitly write the multiplicity for unrestricted functions
explicit :: String %Many -> (String, String); explicit s = (s, s)
-- Unrestricted functions don't typically include a multiplicity annotation
implicit :: String -> (String, String); implicit s = (s, s)
-- Linear types of a multiplicity of %One but %1 is a more common alias
linear :: String %One -> String %1 -> (String, String); linear a b = (a,b)
```

```
-- A multiplicity polymorphic version of ($). 'f' is used with multiplicity
-- 'n', and uses its argument with multiplicity 'm'
polymorphic :: (a %m -> b) %n -> a %m -> b; polymorphic f a = f a
```

It's important to remember that multiplicity annotations are part of the type of an expression. Our puzzle failed because we tried to pass a linear function to `($)`, which expected an unrestricted function. When writing higher-order functions with linear types, you need to consider using multiplicity-polymorphism to write functions that can work with both unrestricted and linear functions.

%One Ring to Rule Them All

After taking the time to understand what linear types are and how they work, the obvious question is, “So what?” In a pure memory-managed language with referential transparency, it's not immediately clear what we gain by trying to ensure that something is used exactly once, or even what it means to “use” a value.

It turns out that each of the rules we've removed from linear types buys us some flexibility for writing efficient pure functions. Weakening in unrestricted types, for example, means that we can potentially lose track of resources we've allocated. Linear types prohibit weakening and make it much harder to forget to release resources that we allocated for a linear API.

Contracting has an even bigger impact. Since unrestricted types allow us to create as many copies of a binding as we want, values must remain pure if we want to preserve referential transparency. Linear types prohibit contracting, and so a well-designed linear API can safely mutate a resource without violating referential transparency since linearity ensures that no other references to the mutated data can exist.

These two properties let us use linear types to carefully build pure APIs around inherently mutable internal resources such as raw memory or foreign objects. As a capstone example for this book, let's review a sketch of a pure functional ring buffer that uses linear types (and makes a few unsafe promises to the compiler) to manage access to a circular buffer of memory-mapped pages. You may not immediately recognize all the underlying foreign functions, but notice how writing the ring buffer consumes the previous value linearly and converting the ring buffer to a `ByteString` hands ownership of the foreign pointer over, allowing the resources to be cleaned up by the foreign pointer finalizer when the `ByteString` is eventually garbage collected.

```

{-# LANGUAGE LinearTypes #-}
{-# LANGUAGE ForeignFunctionInterface #-}
module HaskellBrainTeasers.TimeFliesLikeAnArrow.LinearBuffer where
import Control.Monad
import Prelude.Linear qualified as L
import Data.ByteString qualified as BS
import Data.ByteString.Internal qualified as InternalBS
import Data.Word
import Foreign.C.String
import Foreign.C.Types
import Foreign.Marshal.Utils
import Foreign.Ptr
import GHC.ForeignPtr
import System.Posix.Types
import System.Posix.Internals
import System.IO.Unsafe
import Unsafe.Linear qualified as UL

foreign import ccall "memfd_create" mkMemfd :: CString -> CUInt -> IO FD
foreign import ccall "mmap" mmap ::
  Ptr Word8 -> CSize -> CInt -> CInt -> FD -> COff -> IO (Ptr Word8)
foreign import ccall "munmap" munmap :: Ptr Word8 -> CSize -> IO CInt
foreign import ccall unsafe "getpagesize" getpagesize :: CInt

newtype Aligned = Aligned CInt deriving (Eq, Ord, Enum, Num, Real, Integral)

pageSize :: Aligned
pageSize = fromIntegral getpagesize

data RingBuff where
  RingBuff :: Aligned -> Int -> ForeignPtr Word8 %1 -> RingBuff

withRingBuffer :: Aligned -> (RingBuff %1 -> b) %1 -> b
withRingBuffer size f = f L.$ RingBuff size 0 (unsafeRingBuff size)

unsafeRingBuff :: Aligned -> ForeignPtr Word8
unsafeRingBuff (Aligned alignedSize) = unsafePerformIO $ do
  let
    -- polymorphic size to satisfy different FFI C types
    size :: forall a. Integral a => a
    size = fromIntegral alignedSize
    protoReadWrite = 3
    flagsPrivateAnon = 34
    flagsSharedFixed = 17
    fd <- withCString "ringmapFD" $ \name -> mkMemfd name 0
    void $ c_ftruncate fd size
    region <- mmap nullPtr (2*size) protoReadWrite flagsPrivateAnon (-1) 0
    void $ mmap region size protoReadWrite flagsSharedFixed fd 0
    let overflowRegion = plusPtr region size
    void $ mmap overflowRegion size protoReadWrite flagsSharedFixed fd 0
    void $ c_close fd
    newConcForeignPtr region $ void $ munmap region (2*size)
  {-# NOINLINE unsafeRingBuff #-}

```

```

writeRingBuff :: BS.ByteString -> RingBuff %1 -> RingBuff
writeRingBuff rawInput (RingBuff size offset outputFP) =
  let
    sizedInput = BS.takeEnd (fromIntegral size) rawInput
    (inFP, inOff, inputSize) = InternalBS.toForeignPtr sizedInput
    copySize = min (fromIntegral size) inputSize
    offset' = (offset + copySize) `mod` fromIntegral size
    copyToFP outFP = unsafePerformIO $ do
      withForeignPtr inFP $ \input -> withForeignPtr outFP $ \output ->
        copyBytes (plusPtr output offset) (plusPtr input inOff) copySize
      pure outFP
  in RingBuff size offset' (UL.toLinear copyToFP outputFP)

toByteString :: RingBuff %1 -> BS.ByteString
toByteString (RingBuff size _offset fptr) =
  UL.toLinear InternalBS.fromForeignPtr fptr 0 (fromIntegral size)

```

Further Reading

Multiplicity (1996)

<https://www.themoviedb.org/movie/9304-multiplicity?language=en-US>

Linear base on Hackage

<https://hackage-content.haskell.org/package/linear-base-0.6.0>

Linear types in the GHC User's Guide

https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/linear_types.html

Thank you!

We hope you enjoyed this book and that you're already thinking about what you want to learn next. To help make that decision easier, we're offering you this gift.

Head on over to <https://pragprog.com> right now, and use the coupon code BUYANOTHER2026 to save 30% on your next ebook. Offer is void where prohibited or restricted. This offer does not apply to any edition of *The Pragmatic Programmer* ebook.

	SAVE 30% with coupon code BUYANOTHER2026 when you checkout at pragprog.com
---	--

And if you'd like to share your own expertise with the world, why not propose a writing idea to us? After all, many of our best authors started off as our readers, just like you. With up to a 50% royalty, world-class editorial services, and a name you trust, there's nothing to lose. Visit <https://pragprog.com/become-an-author/> today to learn more and to get started.

Thank you for your continued support. We hope to hear from you again soon!

The Pragmatic Bookshelf

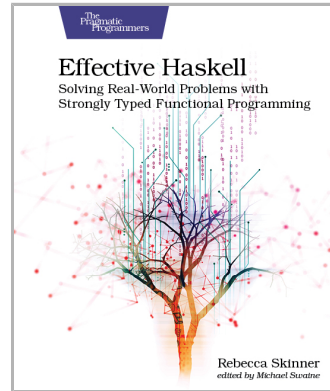
Effective Haskell

Put the power of Haskell to work in your programs, learning from an engineer who uses Haskell daily to get practical work done efficiently. Leverage powerful features like Monad Transformers and Type Families to build useful applications. Realize the benefits of a pure functional language, like protecting your code from side effects. Manage concurrent processes fearlessly. Apply functional techniques to working with databases and building RESTful services. Don't get bogged down in theory, but learn to employ advanced programming concepts to solve real-world problems. Don't just learn the syntax, but dive deeply into Haskell as you build efficient, well-tested programs.

Rebecca Skinner

(668 pages) ISBN: 9781680509342. \$57.95

<https://pragprog.com/book/rshaskell>



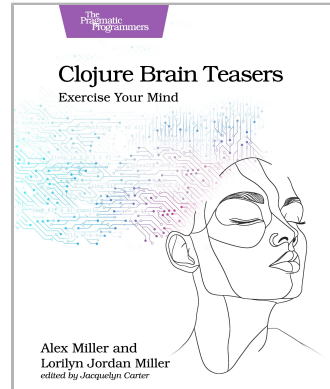
Clojure Brain Teasers

Challenge your knowledge of Clojure with 25 short Clojure teasers, sometimes with surprising results! Inspired by years of developer questions and feedback, these teasers are handpicked to clarify common points of confusion. Each code challenge illustrates Clojure's elegant design, explaining how and why it works. Enjoy these simple exercises solo or with friends to find gaps in your knowledge, challenge assumptions, and gain valuable insights. Tackle the most common points of confusion Clojure developers encounter, become more efficient when writing and debugging, and better predict the outcomes of Clojure code. Regardless of your Clojure experience, you're certain to learn something new.

Alex Miller and Lorilyn Jordan Miller

(130 pages) ISBN: 9798888651292. \$32.95

<https://pragprog.com/book/mmclobrain>



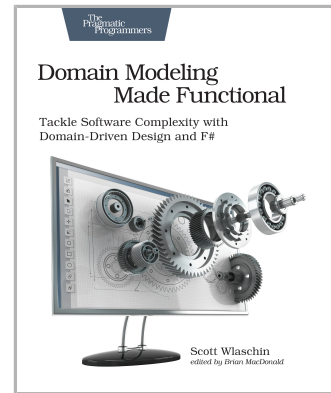
Domain Modeling Made Functional

You want increased customer satisfaction, faster development cycles, and less wasted work. Domain-driven design (DDD) combined with functional programming is the innovative combo that will get you there. In this pragmatic, down-to-earth guide, you'll see how applying the core principles of functional programming can result in software designs that model real-world requirements both elegantly and concisely—often more so than an object-oriented approach. Practical examples in the open-source F# functional language, and examples from familiar business domains, show you how to apply these techniques to build software that is business-focused, flexible, and high quality.

Scott Wlaschin

(310 pages) ISBN: 9781680502541. \$47.95

<https://pragprog.com/book/swdddf>



Web Development with ReasonML

ReasonML is a new, type-safe, functional language that compiles to efficient, readable JavaScript. ReasonML interoperates with existing JavaScript libraries and works especially well with React, one of the most popular front-end frameworks. Learn how to take advantage of the power of a functional language while keeping the flexibility of the whole JavaScript ecosystem. Move beyond theory and get things done faster and more reliably with ReasonML today.

J. David Eisenberg

(208 pages) ISBN: 9781680506334. \$45.95

<https://pragprog.com/book/reasonml>



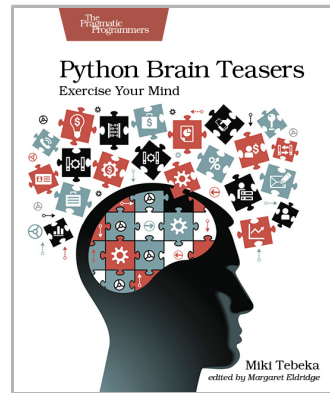
Python Brain Teasers

We geeks love puzzles and solving them. The Python programming language is a simple one, but like all other languages it has quirks. This book uses those quirks as teaching opportunities via 30 simple Python programs that challenge your understanding of Python. The teasers will help you avoid mistakes, see gaps in your knowledge, and become better at what you do. Use these teasers to impress your co-workers or just to pass the time in those boring meetings. Teasers are fun!

Miki Tebeka

(116 pages) ISBN: 9781680509007. \$18.95

<https://pragprog.com/book/d-pybrain>



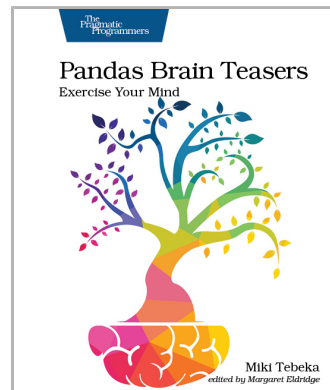
Pandas Brain Teasers

This book contains 25 short programs that will challenge your understanding of Pandas. Like any big project, the Pandas developers had to make some design decisions that at times seem surprising. This book uses those quirks as a teaching opportunity. By understanding the gaps in your knowledge, you'll become better at what you do. Some of the teasers are from the author's experience shipping bugs to production, and some from others doing the same. Teasers and puzzles are fun, and learning how to solve them can teach you to avoid programming mistakes and maybe even impress your colleagues and future employers.

Miki Tebeka

(110 pages) ISBN: 9781680509014. \$18.95

<https://pragprog.com/book/d-pandas>



The Pragmatic Bookshelf

The Pragmatic Bookshelf features books written by professional developers for professional developers. The titles continue the well-known Pragmatic Programmer style and continue to garner awards and rave reviews. As development gets more and more difficult, the Pragmatic Programmers will be there with more titles and products to help you stay on top of your game.

Visit Us Online

This Book's Home Page

<https://pragprog.com/book/haskellbt>

Source code from this book, errata, and other resources. Come give us feedback, too!

Keep Up-to-Date

<https://pragprog.com>

Join our announcement mailing list (low volume) or follow us on Twitter @pragprog for new titles, sales, coupons, hot tips, and more.

New and Noteworthy

<https://pragprog.com/news>

Check out the latest Pragmatic developments, new titles, and other offerings.

Buy the Book

If you liked this ebook, perhaps you'd like to have a paper copy of the book. Paperbacks are available from your local independent bookstore and wherever fine books are sold.

Contact Us

Online Orders: <https://pragprog.com/catalog>

Customer Service: support@pragprog.com

International Rights: translations@pragprog.com

Academic Use: academic@pragprog.com

Write for Us: <http://write-for-us.pragprog.com>